

C++ API FOR

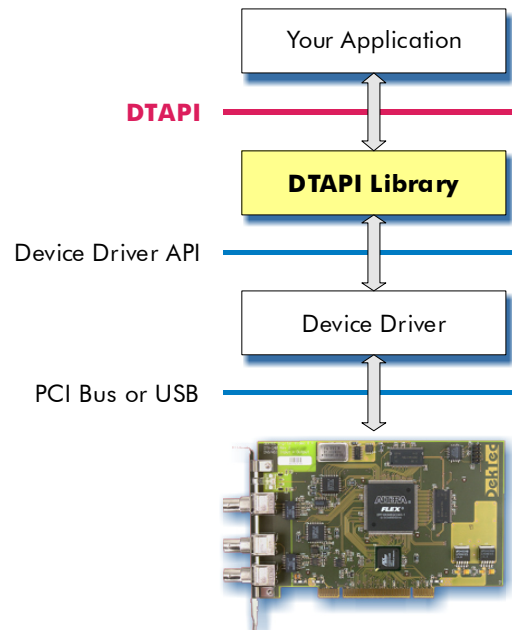
- DTA ADAPTERS FOR PCI
- DTU ADAPTERS FOR USB

- Uniform access to DEKTEC hardware
- Abstracts from hardware details like DMA, USB, Interrupts, VPD
- Object oriented, easy to use

FEATURES

- Common C++ interface to DekTec's line of digital-video devices
- Encapsulates the device-driver layer in an easy-to-use object-oriented interface
- Access to all hardware features from user-mode programs
- High efficiency; no need to resort to kernel-mode programming to achieve real-time operation
- Operates on top of the Windows WDM device driver for Windows 2000 / 2003 / XP
- Packaged in a C++ header file (to be included in your C++ project) and a library (to be linked to the application)

LAYERING



KEY ATTRIBUTES

Parameter	Value
Maximum number of parallel hardware functions per PC	75
DTAPI Version	3.0.5.64
Dta1xx Device-Driver Version	2.0.7.178
Dtu2xx Device-Driver Version	2.6.0.76

SUPPORT – USB ADAPTERS

Type	Description
DTU-205	ASI/SDI Output
DTU-225	ASI/SDI Input
DTU-234	8-VSB/QAM-B Receiver
DTU-245	ASI/SDI Input + Output

SUPPORT – PCI ADAPTERS

Type	Description
DTA-100	DVB-ASI Output
DTA-102	DVB-SPI Output
DTA-105	Dual DVB-ASI Output
DTA-107	QPSK Modulator / Upconverter
DTA-110	QAM Modulator / UHF Upconverter
DTA-110T	OFDM Modulator / UHF Upconverter
DTA-120	DVB-ASI Input
DTA-122	DVB-SPI Input
DTA-124	Quad DVB-ASI Input
DTA-140	DVB-ASI Input+Output
DTA-145	Multipurpose Dual ASI/SDI Ports
DTA-160	GigE TS-over-IP + 3 ASI Ports

Copyright © 2000-2006 by DekTec Digital Video B.V.

DekTec Digital Video B.V. reserves the right to change products or specifications without notice. Information furnished in this document is believed to be accurate and reliable, but DEKTEC Digital Video assumes no responsibility for any errors that may appear in this material.

Table of Contents

1. General Description	5	DtDevice::GetUsbSpeed	45
1.1. Object Model	5	DtDevice::HwFuncScan	46
1.2. Terminology	6	DtDevice::I2CRead	47
1.3. References.....	6	DtDevice::I2CWrite	48
2. Using DTAPI	8	DtDevice::LedControl	49
2.1. Including and Linking DTAPI.....	8	DtDevice::SetIoConfig	50
2.2. Attaching to a DtDevice.....	8	DtDevice::VpdDelete	51
2.3. Attaching to a Channel	9	DtDevice::VpdRead	52
2.4. Initialising a Channel	9	DtDevice::VpdWrite.....	55
2.5. Streaming Data – Input	9		
2.6. Streaming Data – Output	10	DtInpChannel	56
2.7. Using VPD	11	DtInpChannel	56
2.8. Self-Test Support.....	11	DtInpChannel::Attach	57
2.9. Complete Example	11	DtInpChannel::AttachToPort	59
3. DTAPI Classes and Methods	13	DtInpChannel::ClearFifo	60
3.1. Overview.....	13	DtInpChannel::ClearFlags	61
		DtInpChannel::Detach	62
Data Structures	16	DtInpChannel::Equalise	63
Struct DtDeviceDesc	16	DtInpChannel::GetFifoLoad	64
Struct DtHwFuncDesc	18	DtInpChannel::GetFlags.....	65
Struct DtapiHwFunc (Obsolete).....	21	DtInpChannel::GetMaxFifoSize	66
Struct DtRawIpHeader	22	DtInpChannel::GetReceiveByteCount	67
Struct DtTslpPars	23	DtInpChannel::GetRxControl.....	68
		DtInpChannel::GetStatistics.....	69
Global Functions	26	DtInpChannel::GetStatus	70
::DtapiCheckDeviceDriverVersion	26	DtInpChannel::GetTargetId	72
::DtapiGetDeviceDriverVersion	27	DtInpChannel::GetTsRateBps	73
::DtapiGetVersion	28	DtInpChannel::LedControl	74
::DtapiDeviceScan.....	29	DtInpChannel::PolarityControl	75
::DtapiDtDeviceDesc2String	30	DtInpChannel::Read	76
::DtapiDtHwFuncDesc2String	31	DtInpChannel::ReadDirect	77
::DtapiHwFuncScan.....	32	DtInpChannel::Reset	78
::DtapiInitDtTslpParsFromIpString	33	DtInpChannel::SetIpPars	79
::DtapiPciScan (Obsolete).....	34	DtInpChannel::SetLoopBackMode	80
::DtapiResult2Str	35	DtInpChannel::SetPower	81
		DtInpChannel::SetRxControl.....	82
DtDevice	36	DtInpChannel::SetRxMode	83
DtDevice::AttachToSerial.....	36	DtInpChannel::WriteLoopBackData	86
DtDevice::AttachToSlot.....	37		
DtDevice::AttachToType	38	DtOutpChannel	87
DtDevice::Detach	39	DtOutpChannel	87
DtDevice::GetDescriptor.....	40		
DtDevice::GetDeviceDriverVersion	41		
DtDevice::GetFirmwareVersion.....	42		
DtDevice::GetIoConfig	43		
DtDevice::GetRefClkCnt	44		

DtOutpChannel::Attach	88	DtLoop::SetStuffingMode	135
DtOutpChannel::AttachToPort	90	DtLoop::Start	136
DtOutpChannel::ClearFifo	91	4. Ports and Hardware Functions per Device...	137
DtOutpChannel::ClearFlags	92		
DtOutpChannel::Detach	93		
DtOutpChannel::GetExtClkFreq	94		
DtOutpChannel::GetFifoLoad.....	95		
DtOutpChannel::GetFifoSize	96		
DtOutpChannel::GetFlags.....	97		
DtOutpChannel::GetMaxFifoSize.....	99		
DtOutpChannel::GetModControl	100		
DtOutpChannel::GetRfControl	101		
DtOutpChannel::GetTargetId	102		
DtOutpChannel::GetTransmitByteCount	103		
DtOutpChannel::GetTsRateBps	104		
DtOutpChannel::GetTxControl	105		
DtOutpChannel::GetTxMode.....	106		
DtOutpChannel::ReadLoopBackData.....	107		
DtOutpChannel::Reset	108		
DtOutpChannel::SetFifoSize.....	109		
DtOutpChannel::SetIpPars	110		
DtOutpChannel::SetLoopBackMode	111		
DtOutpChannel::SetModControl	112		
DtOutpChannel::SetPower	116		
DtOutpChannel::SetRfControl	117		
DtOutpChannel::SetTsRateBps	118		
DtOutpChannel::SetTxControl.....	119		
DtOutpChannel::SetTxMode.....	121		
DtOutpChannel::Write	123		
DtOutpChannel::WriteDirect	125		
TsInpChannel	126		
TsInpChannel.....	126		
TsOutpChannel	127		
TsOutpChannel.....	127		
DtLoop	128		
DtLoop.....	128		
DtLoop::AttachToInput	129		
DtLoop::AttachToOutput	130		
DtLoop::Detach.....	131		
DtLoop::DetachFromInput	132		
DtLoop::DetachFromOutput	133		
DtLoop::IsStarted	134		

DTAPI Revision History

Version	Date	Change Description
V3.0.5.64	2006.05.22	- New generic classes DtInpChannel and DtOutpChannel with SdiInpChannel, TslpChannel, SdiOutpChannel and DtOutpChannel as derived classes - New concept (physical) Port to identify a specific port of a device; Addition of port-related fields to DtDeviceDesc and DtHwFuncDesc; New member functions DtInpChannel::AttachToPort and DtOutpChannel::AttachToPort - Redefinition of a number of fields in DtHwFuncDesc for improved description of hardware capabilities - New DtLoop class for looping Transport Streams from port to port - New structure DtTslpPars and function DtOutpChannel::SetTslpPars for specifying TS-over-IP parameters - Addition of several string conversion routines - Support for 8-bit SDI on DTA-160, DTU-225 and DTU-245 - Support for DTA-110T, DTA-160, DTU-234, DTU-245
V2.4.4.44	2005.07.11	- Support for DTA-110 - Fixed several errors in the example code included in this document
V2.3.0.39	2004.12.29	New routines: DtInpChannel::GetMaxFifoSize, DtInpChannel::GetReceiveByteCount, DtOutpChannel::GetTransmitByteCount, DtDevice::I2CRead and DtDevice::I2CWrite
V2.1.2.36	2004.09.16	- Support for DTA-107 - Support for DTU-205
V2.0.0.30	2004.03.15	- Added support for DTU-225 - Added DtDevice class
V1.3.1.25	2003.02.07	- Support for time stamping on DTA-140, amongst others addition of time-stamp flag in DtInpChannel::SetRxMode - Add support for receive mode DTAPI_RXMODE_STMP2 - Add support for transmit mode DTAPI_TXMODE_MIN16 - Change of DtInpChannel::Read, so that entire receive FIFO can be read, even without input signal - New routines: PciCard::GetRefClkCnt DtInpChannel::ClearFifo, DtInpChannel::GetRxControl, DtInpChannel::WriteLoopBackData
V1.2.0.20	2002.11.12	- Added this change description section - New routine ::DtapiCheckDeviceDriverVersion
V1.1.0.19	2002.11.11	Support for DTA-140
V1.0.2.6	2002.03.22	New routine DtOutpChannel::ClearFifo to clear transmit FIFO without generation of a truncated packet

1. General Description

DTAPI is an acronym for DekTec Application Programming Interface. The **DTAPI** enables application programs to access the functions of DekTec Devices at a higher level of abstraction than would be possible using direct device-driver calls. Still, the **DTAPI** allows access to nearly all hardware features.

On the Windows™ platform, the **DTAPI** is composed of a header file, to be included by the application source code, and a library file, to be linked to the application's executable.

1.1. Object Model

The **DTAPI** is a collection of C++ classes, each representing a generic hardware function. Instances of a **DTAPI** class refer to a specific hardware function on a particular device. The hardware is controlled indirectly by invoking methods on the **DTAPI** objects.

Figure 1 illustrates a number of instantiated **DTAPI** objects in action. The application interacts with the **DTAPI** objects. The library methods implementing the **DTAPI** classes communicate with the hardware through the WDM device driver.

Each DekTec device is represented by precisely one **DtDevice** object. This object is the starting point for all interaction with a particular DekTec device. The following functions are supported:

- Instantiating channels for streaming.
- Getting versioning information on the device driver and firmware installed on the hardware.
- Reading and writing Vital Product Data (VPD), used for storing product-identification data. VPD is divided in a read-only part (e.g. *part number*) and a read/write part for custom identification information.

The **DtDevice** class is supplemented by global function **DtapiDtDeviceScan** to scan the PCI and USB buses and build an inventory of DekTec devices in the system.

Streaming functions are represented by "channel" objects. For the purpose of streaming MPEG-2 Transport Streams, two channel classes are defined: **DtInpChannel1** for reading

a Transport Stream and **DtOutpChannel1** for outputting a Transport Stream. A device may support multiple physical input and/or output streams, so in general a device can be represented by multiple channel objects.

Class **DtOutpChannel1** supports the following functions:

- Attaching to (and detaching from) the hardware identified by a **DtDevice** object.
- Initialising and controlling the output channel, like setting bit rate and packet size,

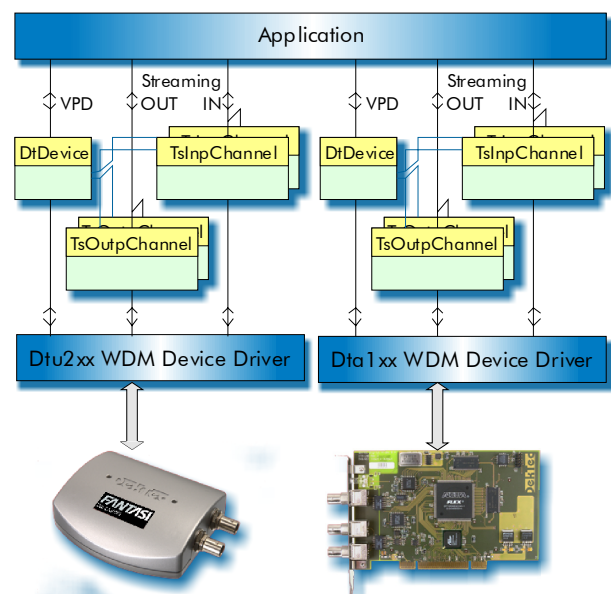


Figure 1. Example of **DTAPI** objects representing two devices.

- reading the load of the Transmit FIFO, etc.
- The core function: writing Transport-Stream data. This function encapsulates DMA details, so that the user is effectively shielded from the entire DMA process.
- Controlling special functions supported by the output channel, such as identification of the connected Target Adapter, applying power supply to the Target Adapter, measurement of external-clock frequency, etc.
- Functions to enable software testing of the Transmit FIFO on-board of the device.

Class **DtInpChannel1** supports similar functions for an input stream:

- Attaching to (and detaching from) the hardware identified by a **DtDevice** object.
- Initialising and controlling the input chan-

nel, like setting packet size, reading bit rate, reading the load of the Receive FIFO, etc.

- Reading incoming Transport-Stream data and storing it in a buffer in host memory.
- Controlling special functions supported by the input channel, such as identification of the connected Target Adapter, applying power supply to the Target Adapter, etc.
- Functions to enable software testing of the Receive FIFO on-board of the device.

Note

- There is no one to one mapping between DTAPI functions and system calls to the device driver: DTAPI method calls may lead to zero, one or more system calls to the device driver.

1.2. Terminology

channel object – Instance of a C++ class representing a single physical input or output stream. The application manipulates input or output streams by invoking methods on the channel object.

DtDevice – DekTec Device, generic designator for either PCI card or USB adapter.

DTA-xxx card – Any DekTec PCI card in the DTA series.

Dta1xx – Name of the WDM device driver for Windows-2000 / XP / 2003. This device driver is generic for DekTec PCI Cards.

DTAPI – DekTec Application Programming Interface.

DTU-xxx device – Any DekTec USB adapter in the DTU series.

Dtu2xx – Name of the WDM device driver for Windows-2000 / XP / 2003. This device driver is generic for DekTec USB Adapters.

(hardware) function – An elementary “capability” of a PCI card, e.g. outputting a Transport Stream in DVB-ASI format. A device can support multiple functions. Note that the interpretation of *function* is not necessarily equivalent to the *PCI-function* concept in the PCI rev. 2.2 specifications.

PCI card – The hardware card that contains the interface-adaptation and/or other processing hardware.

target adapter – External hardware device that translates the DVB-SPI signals on the DTA-102 and DTA-122 to another physical format.

USB Adapter – The USB-compatible hardware that contains the interface-adaptation and/or other processing hardware.

VPD – “Vital Product Data.” Information stored in a PCI device to uniquely identify the hardware and, potentially, software elements of the device. VPD is defined in *PCI Local Bus Specification Rev2.2*. DekTec devices store VPD in on-board serial EEPROMs. DTAPI supports methods to read and write VPD items.

1.3. References

- ISO/IEC 13818-1, *Information technology – Generic coding of moving pictures and associated audio information: Systems*, April 27th, 1995, also known as “MPEG-2 Systems” – Specification of the structure of a MPEG-2 Transport Stream.
- DTA-100, *DVB-ASI Output Adapter for PCI Bus, Product Specification*, DekTec Digital Video B.V., 2002.
- DTA-102, *DVB-SPI Output Adapter for PCI Bus, Product Specification*, DekTec Digital Video B.V., 2002.
- DTA-107, *QPSK Modulator / Upconverter for PCI Bus, Product Specification*, DekTec Digital Video B.V., 2004.
- DTA-120, *DVB-ASI Input Adapter for PCI Bus, Product Specification*, DekTec Digital Video B.V., 2002.
- DTA-122, *DVB-SPI Input Adapter for PCI Bus, Product Specification*, DekTec Digital Video B.V., 2002.
- DTA-124, *Quad ASI/SDI Input Adapter for PCI Bus, Product Specification*, DekTec Digital Video B.V., 2004.
- DTA-140, *DVB-ASI Input+Output Adapter for PCI Bus, Product Specification*, DekTec Digital Video B.V., 2003.
- DTU-205, *DVB-ASI/SDI Output Adapter for USB, Product Specification*, DekTec Digital Video B.V., 2004.

- *DTU-225, DVB-ASI/SDI Input Adapter for USB, Product Specification, DekTec Digital Video B.V., 2004.*
- *PCI Local Bus Specification, Revision 2.2, December 18, 1998 – Formal specification of the protocol, electrical, and mechanical features of the PCI bus.*

2. Using DTAPI

This section discusses the actual usage of the **DTAPI** library. Code snippets are provided to illustrate key methods.

2.1. Including and Linking DTAPI

All **DTAPI** declarations and definitions are contained in one C++ header file: "**DTAPI.h**". Each module that uses **DTAPI** functionality has to include this file.

The **DTAPI** implementation is contained in library file "**DTAPI.lib**", which has to be statically linked to your application.

The **DTAPI** library has been compiled with the C/C++ Code-Generation Options in VC++ set to: "Use run-time library: Multithreaded DLL." On the compiler command line this corresponds to the **/MD** option. Your application has to be compiled with this same setting, or a conflict in the version of the run-time library may occur.

So, to use **DTAPI** follow these steps:

1. Copy files "**DTAPI.h**" and "**DTAPI.lib**" to your project or to a standard location visible to VC++.
2. Add **#include "DTAPI.h"** to each file using **DTAPI** functions.
3. Compile your application using the Multithreaded DLL version of the C run-time library (compiler switch **/MD**).
4. Link statically to "**DTAPI.lib**".

2.2. Attaching to a DtDevice

Programs based on **DTAPI** first have to instantiate a **DtDevice** object and "attach" it to a hardware card. This can be accomplished in several ways.

Method **AttachToType** is convenient when the DekTec device's type number is known and the system contains a single PCI card or USB adapter of the given type¹.

```
DtDevice Dvc;
if (Dvc.AttachToType(122) != DTAPI_OK) {
    // No DTA-122 in the system ...
    // Error-handling code
}
```

Figure 2. Attaching a **DtDevice** object to the hardware PCI card based on type number.

Another, method **AttachToSerial** can be used if the unique serial number of the device is known.

```
DtDevice Dvc;
if (Dvc.AttachToSerial(410000001) !=
    DTAPI_OK) {
    // No card with serial 410000001
    // connected...
    // Error-handling code
}
```

Figure 3. Attaching a **DtDevice** object to the hardware based on serial number.

Alternatively, method **AttachToSlot** can be used if the physical location (PCI Bus#, Slot#) of a PCI card in the system is known.

```
DtDevice Dvc;
if (Dvc.AttachToSlot(1, 3) != DTAPI_OK)
{
    // No card in slot 3 on PCI bus 1 ...
    // Error-handling code
}
```

Figure 4. Attaching a **DtDevice** object to the hardware PCI card based on PCI-Bus and Slot number.

A more sophisticated application creates an inventory of available **DTAPI**-compatible PCI cards and USB Adapters and lets the user configure which device is to be used. Alternatively, the software applies an automatic selection criterion, e.g. the first device with a Transport-Stream output. Such schemes can be built with the help of global functions **DtapiHWFuncScan** and **DtapiDeviceScan**.

¹ Or when usage of the first device of the given type is acceptable.


```

DtHwFuncDesc Funcs[10];
int f, nFuncs;
::DtapiHwFuncScan(10, nFuncs, Funcs);

for (f=0; f<nFuncs; f++) {
    if (Funcs[f].m_ChanType &
        DTAPI_TS_OUTPUT)
        break;
}
if (f == nFuncs) { // No output card }

DtDevice Dvc;
Dvc.AttachToSerial(
    Funcs[f].m_DvcDesc.m_Serial);

```

Figure 5. Selecting the first DtDevice with a Transport-Stream output on board.

After all operations have been completed, a DtDevice object should be detached from the hardware with method **Detach**.

2.3. Attaching to a Channel

Before you can stream data into or out of a DekTec Device, two objects must have been instantiated and attached to the hardware:

- DtDevice object (§2.2);
- DtOutpChannel or DtInpChannel object.

A channel object can be attached to the hardware with the channel's **Attach** member function. This function has two parameters: a pointer to the DtDevice object, identifying the hardware board, and an index, identifying the channel on a multi-channel device. The default value for index is zero, so that for single-channel cards the second parameter needs not be specified.

```

DtDevice Dvc;
// Code to attach to device goes here

DtOutpChannel TsOut;
if (TsOut.Attach(&Dvc) != DTAPI_OK) {
    // Error-handling code
}
TsOut.SetFifoSizeMax();

```

Figure 6. Attaching a DtOutpChannel object to the hardware.

Example code to attach an output channel is shown in Figure 6. After attachment, the size of the channel's Transmit FIFO is initialised to its maximum value.

Input channels are attached to the hardware in precisely the same way, except that the size of the Receive FIFO cannot be programmed.

Just like a DtDevice object, a DtOutpChannel / DtInpChannel object should be detached from the hardware after all operations on the channel have been completed.

2.4. Initialising a Channel

A number of DTAPI methods are available to initialise a channel. Refer to the reference pages later in this document for a description of the semantics.

DtOutpChannel

Method **Reset** clears the contents of the Transmit FIFO and provides other initialization services.

Method **SetTsRateBps** changes the output bit rate.

SetTxMode sets packet size, burst- or continuous mode, and other special modes.

DtInpChannel

Input channels also support a **Reset** method, which clears the Receive FIFO.

Method **SetRxMode** sets the packet size of packets stored in the Receive FIFO.

2.5. Streaming Data – Input

This section considers the actual streaming of data from an external source to a MPEG-2 Transport-Stream processing application. The core of an elementary input-streaming program is shown in Figure 7. This code assumes:

- DtDevice object **Dvc** and DtInpChannel object **TsIn** have been attached to the hardware;
- The Receive FIFO is empty and receive mode has been initialised;
- **ProcessTsData(Buf, NumBytes)** is the user-supplied function that processes **NumBytes** new Transport-Stream data bytes in **Buf**.
- **StopCondition()** is a user-supplied function that returns true if sufficient data has been read.

```
// PRE: Dvc and TsIn attached
char Buf[BUFSIZE];

// Start reading Transport Stream
TsIn.SetRxControl(DTAPI_RXCTRL_RCV);

// Main loop
while (!StopCondition()) {
    TsIn.Read(Buf, BUFSIZE);
    ProcessTsData(Buf, BUFSIZE);
}
```

Figure 7. Processing data from an external Transport-Stream source.

The code is straightforward. First receive mode is set to **Rcv**, which starts reading of incoming data into the Receive FIFO. Then the main loop alternates between reading a buffer load of data and processing the data, until the stop condition becomes true.

The following factors should be considered to achieve optimal results:

- The buffer size (constant **BUFSIZE**) should not be too small. Every data transfer from Receive FIFO to the buffer in host memory incurs non-negligible overhead: a system call invokes the device driver, which for PCI cards, builds a DMA scatter/gather list for every transfer.
A reasonable minimum buffer/transfer size is 4096 bytes. No maximum size exists²; the buffer size may very well be a few Mbytes.
- The number of bytes returned by method **Read** always is a multiple of 4. It is not guaranteed that the data aligns to Transport-Packet boundaries, even if the buffer size is a multiple of the packet size. The processing software should always start with a synchronisation stage.

2.6. Streaming Data – Output

Streaming data to an output is somewhat more involved than inputting data. The core of an elementary output-streaming program is shown in Figure 8. The code assumes:

- **DtDevice** object **Dvc** and **DtOutpChannel** object **TsOut** have been attached to the

hardware;

- Transmit mode has been initialised;
- **GetTsData(Buf, Max)** is the user-supplied function that writes maximally **Max** new Transport-Stream data bytes in **Buf**, and returns the number of bytes written.

Note: The hardware uses 32-bit words internally. Therefore, number of bytes must be a multiple of four at all times.

The first part of the code builds an initial load in the Transmit FIFO before actual transmission begins. Hereto transmission control is set to **Hold**, which enables DMA but keeps transmission disabled.

```
// PRE: Dvc and TsOut attached
TsOut.SetTxControl(DTAPI_TXCTRL_HOLD);

// Build initial load in Transmit FIFO
char Buf[BUFSIZE];
int Load=0;
int NumBytes = GetTsData(Buf, BUFSIZE);
while (Load<INILOAD && NumBytes!=0) {
    TsOut.Write(Buf, NumBytes);
    Load += NumBytes;
    NumBytes = GetTsData(Buf, BUFSIZE);
}
TsOut.SetTxControl(DTAPI_TXCTRL_SEND);

// Main loop
while (NumBytes != 0) {
    TsOut.Write(Buf, NumBytes);
    NumBytes = GetTsData(Buf, BUFSIZE);
}
```

Figure 8. Streaming data to a Transport-Stream output.

When the Transmit FIFO contains its initial load, actual transmission is started by setting transmission control to **Send**. The main loop then supplies additional data to the Transmit FIFO until the data source is exhausted.

The following factors should be considered to achieve optimal results:

- The buffer size (constant **BUFSIZE**) should not be too small. Every data transfer to the Transmit FIFO incurs non-negligible overhead: a system call invokes the device driver, which for PCI cards builds a DMA scatter/gather list for every transfer.

² However, to avoid swapping, the buffer size should not exceed the amount of physical RAM available to a process.

A reasonable minimum buffer/transfer size is 4096 bytes. No maximum size exists³; the buffer size may very well be a few Mbytes.

- The initial Transmit-FIFO load (**INILOAD**) should not be too small either, to prevent an early Transmit-FIFO underflow in the main loop. A value close to the maximum FIFO size is recommended.

Warning. The initial load cannot be larger than the Transmit-FIFO size: when the Transmit FIFO is full, DMA will stall and the application “hangs.”

- As far as **DTAPI** is concerned, the **GetTsData** function may return Transport-Stream data aligned at arbitrary 4-byte boundaries. However, for many data-generating algorithms, alignment on packet boundaries will be a natural choice. In such applications it is convenient and efficient to set the buffer size to a multiple of the packet size.

2.7. Using VPD

Vital Product Data (VPD) is product identification information stored in an EEPROM onboard of DekTec Devices. The read-only part of VPD is loaded in the manufacturing process. The read/write part is used for licensing purposes and for storing customer-specific product information.

VPD is organised as a collection of *items*, each identified by a keyword. Most keywords are 2-character strings (e.g. “PD” for Production Date), with the exception of the VPD ID String, which is identified by “VPDID”.

Three member functions of **DtDevice** are defined to manipulate VPD:

- **VpdRead** – Read VPD-item, given a keyword.
- **VpdWrite** – Write VPD-item, given a keyword and item string. If the item existed, the item string is overwritten, unless the VPD item is read-only.

- **VpdDelete** – Delete VPD item. Read-only VPD item cannot be deleted.

2.8. Self-Test Support

The integrity of memory on DekTec Devices, as used for Input- and Transmit FIFOs, can be tested in *Loop-Back Mode*. In this mode, the FIFO is detached from the card’s input or output and directly accessible to software. This enables construction of a simple memory test: a known pattern is written to the FIFO, after some time the data is read back and compared to the original.

DTAPI-support for self-test is straightforward. Loop-back mode can be entered with method **SetLoopBackMode**. Data can be written to the Receive FIFO with **WriteLoopBackData**. Data can be read from the Transmit FIFO with **ReadLoopBackData**.

2.9. Complete Example

Figure 9 shows the code of a simple but fully functional command-line stream player that is capable of playing out a file. The file name and bit rate at which to play out the file can be specified as command-line arguments.

The example exploits good-old “**stdio**” functions for reading file data. By using a relatively large buffer, performance is more than adequate.

Obviously, this example is just a first step towards a production-quality streamer application. With respect to **DTAPI**, one obvious improvement would be to check the return code for every **DTAPI**-call, and add corresponding error-handling code.

³ However, to avoid swapping, the buffer size should not exceed the amount of physical RAM available to a process.

```

// Command-line program TsOut
// Streams Transport-Stream file out of DTA-100

#define BUFSIZE (1<<16)
#define INILOAD (7*(1<<20))    // 7MB initial load

#include "DTAPI.h"
#include <stdio.h>

int main(int argc, char* argv[])
{
    if (argc != 3) { printf("Usage: TsOut bitrate tsfile\nQuitting...\n"); return -1; }
    // Open Transport-Stream file
    FILE* fp = fopen(argv[2], "rb");
    if (fp == NULL) {
        printf("Can't open '%s' for read\nQuitting...\n", argv[2]);
        return -2;
    }
    // Attach PciCard and output channel objects to hardware
    DtDevice Dvc;
    if (Dvc.AttachToType(100) != DTAPI_OK) {
        printf("No DTA-100 in system. Quitting...\n");
        return -3;
    }
    DtOutpChannel TsOut;
    if (TsOut.Attach(&Dvc) != DTAPI_OK) {
        printf("Can't attach output channel.\nQuitting...\n");
        return -4;        // Detach is executed from DtDevice destructor
    }
    // Initialise bit rate and packet mode
    TsOut.SetTsRateBps(DTAPI_TXCLOCK_INTERNAL, atoi(argv[1]));
    TsOut.SetTxMode(DTAPI_TXMODE_188, 1);

    // Build initial load in Transmit FIFO
    TsOut.SetTxControl(DTAPI_TXCTRL_HOLD);
    char Buf[BUFSIZE];
    int Load = 0;
    int NumBytes = fread(Buf, 1, BUFSIZE, fp);
    while (Load<INILOAD && NumBytes!=0) {
        TsOut.Write(Buf, NumBytes);
        Load += NumBytes;
        NumBytes = fread(Buf, 1, BUFSIZE, fp);
    }
    // Start transmission
    TsOut.SetTxControl(DTAPI_TXCTRL_SEND);
    // Main loop
    while (NumBytes != 0) {
        TsOut.Write(Buf, NumBytes);
        NumBytes = fread(Buf, 1, BUFSIZE, fp);
    }
    TsOut.Detach (DTAPI_INSTANT_DETACH);
    return 0;        // Detach is executed from DtDevice destructor
}

```

Figure 9. Complete command-line application to stream a file with the DTA-100.

3. DTAPI Classes and Methods

3.1. Overview

Table 1. **DTAPI** – Global Functions

API Function	Description
::DtapiCheckDeviceDriverVersion	Check compatibility between Dta1xx and DTAPI
::DtapiGetDeviceDriverVersion	Get version of DTA1xx device driver
::DtapiGetVersion	Get version of DTAPI library
::DtapiDeviceScan	Scan for DTAPI devices
::DtapiHWFuncScan	Scan for DTAPI hardware functions
::DtapiPciScan (obsolete)	Scan DTAPI hardware functions on PCI bus

Table 2. **DTAPI** – **DtDevice** Functions
(the DtDevice Functions are also accessible as PCICard Functions for backwards compatibility with DTAPI version 1.3)

API Function	Description
AttachToSerial	Attach DtDevice object to hardware, based on serial number
AttachToSlot	Attach DtDevice object to hardware, based on slot number
AttachToType	Attach DtDevice object to hardware, based on type number
Detach	Detach DtDevice object from hardware
GetDeviceDriverVersion	Get version of device driver for this DtDevice
GetRefClkCnt	Get sample of reference clock counter
GetFirmwareVersion	Get version of the firmware loaded on the device
I2CRead	Read data from the I2C bus
I2CWrite	Write data to the I2C bus
LedControl	Control of general-status LED
VpdDelete	Delete Vital-Product-Data item
VpdRead	Read Vital-Product-Data item

VpdWrite	Write Vital-Product-Data item
----------	-------------------------------

Table 3. **DTAPI** – **DtInpChannel1** Functions

API Function	Description
Attach	Attach channel object to hardware
ClearFifo	Clear receive FIFO
ClearFlags	Clear latched status flag(s)
Detach	Detach channel object from hardware function
Equalise	Control DVB-ASI cable equalisation (DTA-120 Rev 1 only).
GetFifoLoad	Get current load of the Transmit FIFO
GetFlags	Get status flags
GetRxControl	Get receive-control state
GetStatistics	Get statistics information
GetStatus	Get status information
GetMaxFifoSize	Get the size of the Receive FIFO
GetReceiveByteCount	Get a sample from the free running 32-bit received number of bytes counter
GetTargetId	Get target-adaptor identifier (DTA-122 only)
GetTsRateBps	Get an estimate of the input Transport-Stream rate
LedControl	Control of input-status LED
PolarityControl	Control polarity-detection circuitry of a DVB-ASI input
Read	Read input data
ReadDirect	Read input data without DMA
Reset	Reset input channel
SetLoopBackMode	Set loop-back mode
SetRxPower	Control power for target a-dapter (DTA-122)
SetRxControl	Set receive-control state
SetRxMode	Set receive mode

WriteLoopBack Data	Write data to Receive FIFO via loop-back channel
---------------------------	--

Table 4. **DTAPI** – **DtOutpChannel** Functions

API Function	Description
Attach	Attach channel object to hardware
ClearFifo	Clear transmit FIFO
ClearFlags	Clear latched status flag(s)
Detach	Detach channel object from hardware function
GetExtClkFreq	Get external-clock frequency (DTA-102 only)
GetFifoLoad	Get current load of the Transmit FIFO
GetFlags	Get status flags
GetMaxFifoSize	Get maximum size of Output-FIFO
GetModControl	Get modulation parameters
GetRfControl	Get RF parameters
GetTargetId	Get target-adaptor identifier (DTA-102 only)
GetTransmitByte Count	Get a sample from the free running 32-bit transmitted number of bytes counter
GetTsRateBps	Get Transport-Stream rate

Table 5. **DTAPI** – **DtLoop** Functions

API Function	Description
AttachToInput	Attach to an DtInpChannel object
AttachToOutput	Attach to an DtOutpChannel object
Detach	Detach from the associated DtInpChannel and DtOutpChannel objects
DetachFrom-Input	Detach from the associated DtInpChannel object
DetachFrom-Output	Detach from the associated DtOutpChannel object
IsStarted	Has looping started
SetSuffingMode	Set the NULL-packet stuffing parameters

GetTxControl	Get transmit-control state (Idle/Hold/Send)
GetTxMode	Get transmit mode (packet size, packet-stuffing mode)
ReadLoopBack-Data	Read data from loop-back channel
Reset	Reset output channel
SetFifoSize	Set size of Transmit FIFO
SetFifoSizeMax	Set size of Transmit FIFO to its maximum value
SetLoopBack-Mode	Set loop-back mode
SetModControl	Set modulation parameters
SetPower	Control power for target adapter (DTA-102)
SetRfControl	Set RF parameters
SetTsRateBps	Set Transport-Stream rate
SetTxControl	Set transmit-control state (Idle/Hold/Send)
SetTxMode	Set transmit mode (packet size, ...) and packet stuffing on/off
Write	Write data to the output channel
WriteDirect	Write data directly to the output channel without using DMA

Start	Start/Stop looping from input-to-output
-------	---

Data Structures

Struct DtDeviceDesc

Structure describing a DekTec device.

```
struct DtDeviceDesc {
    int      m_Category;           // Device category
    __int64  m_Serial;            // Unique serial number of the device
    int      m_PciBusNumber;      // PCI-bus number
    int      m_SlotNumber;       // PCI-slot number
    int      m_UsbAddress;       // USB address
    int      m_TypeNumber;       // Device type number
    int      m_DeviceId;         // Device ID
    int      m_VendorId;         // Vendor ID
    int      m_SubsystemId;      // Subsystem
    int      m_SubVendorId;      // Subsystem Vendor ID
    int      m_NumHwFuncs;       // #Hardware funtions hosted by device
    int      m_FirmwareVersion;  // Firmware version
    int      m_FirmwareVariant;  // Firmware variant
    int      m_NumDtInpChan;     // Number of input channels
    int      m_NumDtOutpChan;    // Number of output channels
    int      m_NumPorts;         // Number of physical ports
};
```

Members

m_Category

Code indicating the device category.

Value	Meaning
DTAPI_CAT_PCI	PCI Bus device
DTAPI_CAT_USB	USB device

m_Serial

The serial number that uniquely identifies a DekTec device.

m_PciBusNumber, m_SlotNumber

If the device belongs to category **DTAPI_CAT_PCI**, these integers identify the PCI bus and slot number in which the PCI card is installed. If the device belongs to another category, the values of these members are undefined.

m_UsbAddress

If the device belongs to category **DTAPI_CAT_USB**, this number identifies the USB address of the device. If the device belongs to another category, the value of this member is undefined.

m_TypeNumber

This integer corresponds to the number in the device's type string, e.g. 100 for the DTA-100.

m_DeviceId, m_VendorId, m_SubsystemId, m_SubVendorId

Device ID, Vendor ID, Subsystem ID and Subsystem Vendor ID. Identification information of the device, as read from its configuration-space registers.

m_NumHwFuncs

Number of hardware functions hosted by the device.

m_FirmwareVersion

Version number of the firmware loaded on the device.

m_FirmwareVariant

Variant of the firmware loaded on the device. Some devices may support multiple variants of the firmware each with different functionality.

m_NumDtInpChan

Number of input channels available on the device.
IP ports count as 1 input channel and 1 output channel.

m_NumDtOutpChan

Number of output channels available on the device.
IP ports count as 1 input channel and 1 output channel.

m_NumPorts

Number of physical ports available on the device.
Doubly-buffered outputs count as a single port.

Struct DtHwFuncDesc

Structure describing a hardware function.

```
struct DtHwFuncDesc {
    DtDeviceDesc    m_DvcDesc;           // Device descriptor
    int             m_ChanType;          // Channel type (OR-able)
    int             m_StreamType;        // Stream type
    int             m_Flags;             // Capability flags (OR-able)
    int             m_IndexOnDvc;        // Relative index for this function
    int             m_Port;              // Physical port number
    int             m_Ip[4];             // IP address
    int             m_MacAddr[6];        // MAC address
};
```

Members

m_DvcDesc

Structure describing the device that hosts this hardware function.

m_ChanType

This member variable identifies the channel type of the hardware function. Channel types may be OR-ed together, for TS-over-IP channels both **DTAPI_CHAN_INPUT** and **DTAPI_CHAN_OUTPUT** are set.

Value	Channel Object	Meaning
DTAPI_CHAN_INPUT	DtInpChannel	Input channel
DTAPI_CHAN_OUTPUT	DtOutpChannel	Output channel

m_StreamType

This member variable describes the type of stream type supported by the hardware function.

Value	Meaning
DTAPI_ASI_SDI	DVB-ASI and/or SDI Member variable <i>m_Flags</i> indicates whether the TS interface is capable of ASI, SDI or both
DTAPI_TS_MOD	Modulated signal Member variable <i>m_Flags</i> provides details about the modulation standards that are supported
DTAPI_TS_OVER_IP	TS-over-IP
DTAPI_TS_SPI	DVB-SPI

m_Flags

Extra flags that provide further information about the hardware function.

If *m_StreamType* is **DTAPI_ASI_SDI**, the following OR-able flags are supported:

Value	Meaning
DTAPI_CAP_ASI	Interface can carry DVB-ASI
DTAPI_CAP_BIDIR	Port is bi-directional
DTAPI_CAP_DBLBUF	Port can act as a doubly-buffered copy of another port
DTAPI_CAP_SDI	Interface can carry SDI (Serial Digital Interface)

If *m_StreamType* is **DTAPI_TS_MOD**, the following OR-able flags specify the type(s) of modulation that are supported:

Value	Meaning
DTAPI_CAP_LBAND	L-Band 950 – 2150 MHz
DTAPI_CAP_OFDM	OFDM, as used in DVB-T and DVB-H
DTAPI_CAP_QAM_A	QAM, ITU-T J.83 Annex A (DVB-C)
DTAPI_CAP_QAM_B	QAM, ITU-T J.83 Annex B (US)
DTAPI_CAP_QAM_C	QAM, ITU-T J.83 Annex C (Japan)
DTAPI_CAP_QPSK	QPSK, as used in DVB-S
DTAPI_CAP_UHF	UHF Band 400 – 862 MHz
DTAPI_CAP_VHF	UHF Band 47 – 470 MHz
DTAPI_CAP_8VSB	8-VSB modulated signal

m_IndexOnDvc

This integer identifies a specific hardware function when the device hosts multiple hardware functions with the same channel type and stream type.

If the function occurs only once, *m_IndexOnDvc* = 0. If the device supports the function twice, indices are 0 and 1; etc.

m_Port

This integer identifies the physical port number associated with this function.

The general rule on PCI cards is that the top-most port is #1, the one below that #2, etc., with the following exceptions:

- The Ethernet port on the DTA-160 (the top-most port) has port #4;
- Doubly-buffered outputs like on the DTA-140 count as a single port.

Please refer to Section 0 for an overview of port numbers per DekTec device.

m_Ip

IP address of the hardware function.

NOTE: this field is only valid for functions for which *m_StreamType* is **DTAPI_TS_OVER_IP**.

m_Mac

MAC address of the hardware function.

NOTE: this field is only valid for functions for which *m_StreamType* is **DTAPI_TS_OVER_IP**.

Remarks

This structure is used by `::DtapiHwFuncScan` to return a description of a hardware function.

The channel type of bi-directional ASI/SDI ports (capability `DTAPI_CAP_BIDIR` is set) is *either* `DTAPI_TS_INPUT` or `DTAPI_TS_OUTPUT`. Method `DtDevice::SetIoConfig` can be used to change the direction. The next time `::DtapiHwFuncScan` is called, the channel type in the hardware-function descriptor will be updated to reflect the last-programmed direction.

Struct DtapiHwFunc (Obsolete)

Structure that describes a hardware function on a PCI card.

```
struct DtapiHwFunc {
    int    m_nPciBusNumber;    // PCI-bus number
    int    m_nSlotNumber;     // PCI-slot number
    int    m_nTypeNumber;     // PCI-card type number
    int    m_nDeviceId;       // Device ID of PCI card
    int    m_nVendorId;       // Vendor ID of PCI card
    int    m_nSubsystemId;    // Subsystem ID of PCI card
    int    m_nSubVendorId;    // Subsystem Vendor ID of PCI card
    int    m_nHwFuncType;     // Hardware-function type
    int    m_nHwFuncFlags;    // Hardware-function flags
    int    m_nIndexOnCard;    // Index of hardware function
};
```

Members

m_nPciBusNumber, m_nSlotNumber

Integers identifying the PCI-bus and the slot number in which the PCI card hosting the hardware function is plugged.

m_nTypeNumber

This integer corresponds to the number in the PCI card's type string, e.g. 100 for the DTA-100.

m_nDeviceId, m_nVendorId, m_nSubsystemId, m_nSubVendorId

Device ID, Vendor ID, Subsystem ID and Subsystem Vendor ID. Identification information of the PCI card read from its PCI configuration-space registers.

m_nHwFuncType

Identifies the hardware function according to the table below.

Value	Meaning
DTAPI_TS_INPUT	Transport-Stream input.
DTAPI_TS_OUTPUT	Transport-Stream output.

m_nHwFuncFlags

Hardware-function dependent flags that provide further (static) information about the hardware function. Not used (yet) for any of the current DekTec boards.

m_nIndexOnCard

This number identifies a specific hardware function when the PCI card contains multiple instantiations of the same hardware function. If the function occurs only once, *m_nIndexOnCard* = 0. If the card contains a function twice, the indices are 0 and 1; etc.

Remarks

This structure was targeted at PCI card. It has been superseded by **DtHwFuncDesc**, which supports the more generic notion of a (digital-video) "device".

Struct DtRawIpHeader

Structure placed in front of all IP Packets when **DTAPI_RXMODE_IPRAW** mode is used (refer to **DtInpChannel::SetRxMode** on page 83).

```
struct DtRawIpHeader {  
    unsigned short    m_Tag;           // 0x44A0h = 'D'160  
    unsigned short    m_Length;       // IP Packet Length  
    unsigned long     m_TimeStamp;    // IP Packet arrival time  
};
```

Members

m_Tag

Marks the beginning of a **DtRawIpHeader** structure. The value of this field is fixed to: 0x44A0.

m_Length

Indicates the number of bytes (i.e. size of the IP packet) following directly after the **DtIpRawHeader** structure.

m_TimeStamp

A 32-bit time stamp, taken from the internal system clock on the device, indicating the arrival time of the IP packet following this structure.

Struct DtTsIpPars

Structure for storing parameters related to the transmission of Transport Streams over IP.

The parameters in this structure are used when starting transmission of a Transport Stream over IP to a unicast or multicast destination (**Tx**), and when starting reception of a Transport Stream (**Rx**). This structure is also used for reading back parameters.

```
struct DtTsIpPars {
    u_char    m_Ip[4];           // IP address
    u_short   m_Port;           // Port number
    u_char    m_SrcFltIp[4];    // Source filter: IP address
    u_short   m_SrcFltPort;     // Source filter: port number
    int       m_MulticastTtl;    // TTL for multicast Tx
    int       m_NumTpPerIp;     // #TPs per IP packet
    int       m_Protocol;       // Protocol: UDP/RTP
    int       m_DiffServ;       // Differentiated Services
    int       m_FecMode;        // Error correction mode
    int       m_FecNumRows;     // 'D' = #rows in FEC matrix
    int       m_FecNumCols;     // 'L' = #columns in FEC matrix
    int       m_Flags;          // Optional controls/status flags
};
```

Members (Rx)

m_Ip

Main IP Address. IP address from which to receive IP packets. If the IP address is in the multicast range, the DTAPI automatically joins and drops membership of the multicast group.

m_Port

Port number at which to receive IP packets. Destination port number 0 is not allowed. When the protocol is RTP, the port number shall be even.

m_SrcFltIp

Source-Filter IP Address. Relevant for multicast reception only: In this case *m_SrcFltIp* can be set to a specific IP address for listening to a single source, or to 0.0.0.0 if the source may be any IP address.

m_SrcFltPort

Source-Filter Port number. Port number associated with *m_SrcFltIp*. *m_SrcPort* may be set to a specific source port number, or to 0 for accepting IP packets from any source port.

m_MulticastTtl

Not used.

m_NumTpPerTp

Not used.

Read back: Number of Transport Packets (TPs) stored in one IP packet in the incoming stream.

m_Protocol

Protocol expected for encapsulation of Transport Packets.

Read back: set to the protocol that has been detected.

Value	Meaning
DTAPI_PROTO_UDP	UDP
DTAPI_PROTO_RTP	RTP
DTAPI_PROTO_AUTO	Automatically detect UDP or RTP

m_DiffServ

Not used.

m_FecMode

Error-correction mode.

Value	Meaning
DTAPI_FEC_DISABLE	Don't try to apply error correction
DTAPI_FEC_2D	Try to apply error correction with the FEC streams received on port numbers <i>m_Port+2</i> and <i>m_Port+4</i>

m_FecNumRows, m_FecNumCols

Not used.

m_Flags

Optional control/status flags field. Currently this field is not used and should be set to zero.

Members (Tx)

m_Ip, m_Port

IP address used for transmission, specified as 4 bytes, and associated destination port number. When the protocol is RTP, the port number shall be even.

m_SrcFltIp, m_SrcFltPort

Not used for specifying Tx parameters.

Read back: The host's IP address is stored in *m_SrcFltIp* and the selected source port number in *m_SrcFltPort*.

m_MulticastTtl

Time-To-Live (TTL) value to be used for multicast transmission. When *m_MulticastTtl* is 0, a default value is used.

m_NumTpPerIp

Number of Transport Packets (TPs) stored in one IP packet. The valid range is 1..7.

m_Protocol

Protocol expected for encapsulation of Transport Packets.

Value	Meaning
DTAPI_PROTO_UDP	UDP
DTAPI_PROTO_RTP	RTP

m_DiffServ

Value to be put in the *Differentiated Services* field (formerly *Service Type*) in the IP header.

m_FecMode

Error-correction mode.

Value	Meaning
DTAPI_FEC_DISABLE	No FEC
DTAPI_FEC_2D	RFC2733 parity FEC with 2D extensions as described in Code of Practice #3

m_FecNumRows, m_FecNumCols

Number of rows and columns in the FEC matrix. In the COP #3 these parameters are called *D* and *L* respectively. The following restrictions apply to *L* and *D*:

$$4 \leq D \leq 20, \quad 1 \leq L \leq 20 \quad \text{and} \quad L * D \leq 100$$

m_Flags

Optional control/status flags field. Currently this field is not used and should be set to zero.

Remarks

Global Functions

::DtapiCheckDeviceDriverVersion

Check whether the versions of the device drivers are compatible with the current version of the DTAPI library.

```
DTAPI_RESULT  ::DtapiCheckDeviceDriverVersion (void);
```

Parameters

Result

DTAPI_RESULT	Meaning
DTAPI_OK	The device drivers are compatible with the current version of the DTAPI library.
DTAPI_E_DRIVER_INCOMP	Version of at least one of the device drivers is incompatible with the DTAPI library. The device driver(s) needs to be upgraded.
DTAPI_E_NO_DEVICE	Device-driver version cannot be queried because no DekTec device is installed in the system.

Remarks

::DtapiGetDeviceDriverVersion

Get device-driver version information. Two overloaded variants of this function are provided. The first variant is compatible with DTAPI v1.3.x.x and returns the version of the PCI-card driver (*Dta1xx*). The second variant includes a parameter to specify the device-driver category, either PCI (device driver *Dta1xx*) or USB (device driver *Dtu2xx*).

```
DTAPI_RESULT  ::DtapiGetDeviceDriverVersion (
    [out] int&  DriverVersionMajor,
    [out] int&  DriverVersionMinor,
    [out] int&  DriverVersionBugFix,
    [out] int&  DriverVersionBuild
);
DTAPI_RESULT  ::DtapiGetDeviceDriverVersion (
    [in] int  DvcCategory,          // Device-driver category (PCI/USB)
    [out] int&  DriverVersionMajor,
    [out] int&  DriverVersionMinor,
    [out] int&  DriverVersionBugFix,
    [out] int&  DriverVersionBuild
);
```

Parameters

DvcCategory

Parameter specifying the device category:

Value	Meaning
DTAPI_CAT_PCI	PCI-bus device; Query version of <i>Dta1xx</i> device driver.
DTAPI_CAT_USB	USB device; Query version of <i>Dtu1xx</i> device driver.

DriverVersionMajor

Major version number of the device driver. This number is incremented for major functional upgrades of the device driver.

DriverVersionMinor

The minor version number is incremented for small functional increments of the device driver.

DriverVersionBugFix

The bug-fix version number is incremented when a bug in the device driver has been fixed, without further functional enhancements to the driver.

DriverVersionBuild

Build number.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Device-driver version has been retrieved successfully.
DTAPI_E_NO_DEVICE	Device-driver version cannot be queried because no DekTec device is installed in the system.

Remarks

::DtapiGetVersion

Get version of the **DTAPI** library. Two overloaded variants are defined. The first function prototype, which is defined for backward compatibility, returns the major and minor version number. The second variant also provides bug fix and build number.

```
DTAPI_RESULT ::DtapiGetVersion (
    [out] int& LibVersionMajor,
    [out] int& LibVersionMinor
);

DTAPI_RESULT ::DtapiGetVersion (
    [out] int& LibVersionMajor,
    [out] int& LibVersionMinor,
    [out] int& LibVersionBugFix,
    [out] int& LibVersionBuild
);
```

Parameters

LibVersionMajor

Major version number of the **DTAPI** library. This number is incremented for major functional upgrades of the **DTAPI**.

LibVersionMinor

The minor version number is incremented for small functional increments of the **DTAPI**.

LibVersionBugFix

This number is incremented when a bug in the **DTAPI** library has been fixed, without functional enhancements.

LibVersionBuild

Build number.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Version numbers have been retrieved successfully.

Remarks

This function always succeeds.

The device drivers have their own version number(s), independent from the **DTAPI** library version.

::DtapiDeviceScan

Scan DekTec devices in the system.

```
DTAPI_RESULT ::DtapiDeviceScan (
    [in] int    NumEntries,           // #Device entries in DvcDescArr
    [out] int&   NumEntriesResult,    // #Devices found or #entries required
    [out] DtDeviceDesc* DvcDescArr    // Device descriptor array
);
```

Parameters

NumEntries

Specifies the size, in number of **DtDeviceDesc** entries, of the caller-supplied *pDvcDesc* array.

NumEntriesResult

Output parameter that receives the number of devices found and described in *pDvcDesc*. The value of this parameter can be greater than *NumEntries* (when **DtapiDeviceScan** returns **DTAPI_E_BUF_TOO_SMALL**).

DvcDescArr

Pointer to a caller-supplied array of **DtDeviceDesc** entries to receive the device descriptions.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Scan has completed successfully and the <i>pDvcDesc</i> array was large enough to contain all device descriptions.
DTAPI_E_BUF_TOO_SMALL	The number of device-description entries in <i>pDvcDesc</i> is too small. The number of entries required is returned in <i>NumEntriesResult</i> .

Remarks

DtapiDeviceScan scans the PCI and USB bus(es) in the current system and returns all DekTec devices found.

This function may have to be called twice. The first time, *NumEntries* should be set to a best-guess maximum value. If the result status is **DTAPI_E_BUF_TOO_SMALL**, the application should free the current array of **DtDvcDesc** entries, allocate a new array with the number of entries returned in *NumEntriesResult*, and call **DtapiDeviceScan** again.

::DtapiDtDeviceDesc2String

Creates a descriptive string, e.g. "DTA-100 in Slot 5" for a device.

```
DTAPI_RESULT ::DtDeviceDesc2String (
    [in] DtDeviceDesc* pDvcDesc,    // Device descriptor
    [in] int StringType,           // Type of string to create
    [out] char* pString             // String buffer
    [in] int StringLength          // Size of the string buffer
);

DTAPI_RESULT :: DtDeviceDesc2String (
    [in] DtDeviceDesc* pDvcDesc,    // Device descriptor
    [in] int StringType,           // Type of string to create
    [out] wchar_t* pString          // String buffer
    [in] int StringLength          // Size of the string buffer
);
```

Parameters

pDvcDesc

Pointer to the hardware function descriptor used as input to create our string description.

StringType

Defines the type of string to create. Can be any of the values defined in the table below. The values should be prefixed by **DTAPI_DVC2STR_**.

Value	Example	Meaning
TYPE_NMB	"DTA-100"	Device type number
TYPE_AND_LOC	"DTA-102 in Slot 5"	Device type number and location

pString

Pointer to the buffer that receives the descriptive string

StringLength

Size of the provided buffer (including space for '\0' termination). If the size specified here is too short, the generated string will be clipped and no error is returned. A size of 64 characters should suffice for all strings created with this function.

Results

DTAPI_RESULT	Meaning
DTAPI_OK	Successfully created a string
DTAPI_E_INVALID_BUF	An invalid buffer pointer is supplied for <i>pDvcDesc</i> or <i>pString</i>

::DtapiDtHwFuncDesc2String

Creates a descriptive string, e.g. "DTA-100 in Slot 5" or "DVB-ASI", for a hardware function based on a DtHwFuncDesc structure.

```
DTAPI_RESULT ::DtapiDtHwFuncDesc2String (
    [in] DtHwFuncDesc* pHwFunc,      // Hardware function descriptor
    [in] int StringType,             // Type of string to create
    [out] char* pString              // String buffer
    [in] int StringLength            // Size of the string buffer
);

DTAPI_RESULT ::DtapiDtHwFuncDesc2String (
    [in] DtHwFuncDesc* pHwFunc,      // Hardware function descriptor
    [in] int StringType,             // Type of string to create
    [out] wchar_t* pString           // String buffer
    [in] int StringLength            // Size of the string buffer
);
```

Parameters

pHwFunc

Pointer to the hardware function descriptor used as input to create our string description.

StringType

Defines the type of string to create. Can be any of the values defined in the table below. The values should be prefixed by `DTAPI_HWF2STR_`.

Value	Example	Meaning
<code>TYPE_NMB</code>	"DTA-100"	Device type number
<code>TYPE_AND_PORT</code>	"DTA-124 port 1"	Device type number and port number
<code>TYPE_AND_LOC</code>	"DTA-102 in Slot 5"	Device type number and location
<code>ITF_TYPE</code>	"DVB-ASI"	Physical interface type
<code>ITF_TYPE_SHORT</code>	"ASI"	Physical interface type – short descriptive string

pString

Pointer to the buffer that receives the descriptive string

StringLength

Size of the provided buffer (including space for '\0' termination). If the size specified here is too short, the generated string will be clipped and no error is returned. A size of 64 characters should suffice for all strings created with this function.

Results

DTAPI_RESULT	Meaning
<code>DTAPI_OK</code>	Successfully created a string
<code>DTAPI_E_INVALID_BUF</code>	An invalid buffer pointer is supplied for <i>pHwFunc</i> or <i>pString</i>

::DtapiHwFuncScan

Scan hardware functions hosted by DekTec devices.

```
DTAPI_RESULT ::DtapiHwFuncScan (
    [in] int    NumEntries,           // #Function entries in pHwFuncs
    [out] int&   NumEntriesResult,    // #Functions found or #entries required
    [out] DtHwFuncDesc* pHwFuncs     // Hardware-function descriptions
);
```

Parameters

NumEntries

Specifies the size, in number of **DtHwFuncDesc** entries, of the caller-supplied *pHwFuncs* array.

NumEntriesResult

Output parameter that receives the number of hardware functions found and described in *pHwFuncs*. The value of this parameter can be greater than *NumEntries* (when **DtapiHwFuncScan** returns **DTAPI_E_BUFFER_TOO_SMALL**).

pHwFuncs

Pointer to a caller-supplied array of **DtHwFuncDesc** entries to receive the hardware-function descriptors.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Scan has completed successfully and the <i>pHwFuncs</i> array was large enough to contain all function descriptions.
DTAPI_E_BUF_TOO_SMALL	The number of function-description entries in <i>pHwFuncs</i> is too small. The number of entries required is returned in <i>NumEntriesResult</i> .

Remarks

DtapiHwFuncScan scans the PCI and USB bus(es) in the current system and returns all hardware functions hosted by DekTec devices. Each device may host multiple hardware functions.

This function may have to be called twice. The first time, *NumEntries* should be set to a best-guess maximum value. If the result status is **DTAPI_E_BUF_TOO_SMALL**, the application should free the current array of **DtHwFuncDesc** entries, allocate a new array with the number of entries returned in *NumEntriesResult*, and call **DtapiHwFuncScan** again.

The hardware-function descriptors are always retrieved in the same order. Hardware functions hosted by the same device are grouped together. Within a group of hardware functions hosted by a particular device, functions of the same type are grouped together. These sequencing rules enable application programs to easily create function lists in a 'logical' order.

::DtapiInitDtTsIpParsFromIpString

Initializes the `m_DstIp` and `m_SrcIp` members of a `DtTsIpPars` structure.

```
DTAPI_RESULT ::DtapiInitDtTsIpParsFromIpString (
    [out] DtTsIpPars&  TsIpPars,          // TsIpPars structure to initialize
    [in] const char*   pDstIp,            // String with destination IP
    [in] const char*   pSrcIp             // String with source IP
);

DTAPI_RESULT ::DtapiInitDtTsIpParsFromIpString (
    [out] DtTsIpPars&  TsIpPars,          // TsIpPars structure to initialize
    [in] const wchar_t* pDstIp,           // String with destination IP
    [in] const wchar_t* pSrcIp            // String with source IP
);
```

Parameters

TsIpPars

Ts-IP-Parameter structure to initialize.

pDstIp

Pointer to a string that holds the IP address (e.g. "127.0.0.1") to be used as destination IP address (`m_DstIp`). If this pointer is NULL the IP address "0.0.0.0" will be used.

pSrcIp

Pointer to a string that holds the IP address (e.g. "192.168.0.1") to be used as source IP address (`m_SrcIp`). If this pointer is NULL the IP address "0.0.0.0" will be used.

Results

DTAPI_RESULT	Meaning
DTAPI_OK	This method cannot fail

Remarks

This method only initializes the `m_DstIp` and `m_SrcIp` members, it will leave the other members untouched.

::DtapiPciScan (Obsolete)

Scan hardware functions available on the DTA-xxx cards in the current system.

```
DTAPI_RESULT ::DtapiPciScan (
    [in] int    NumFuncEntries,           // #Function entries in pHwFuncs
    [out] int& NumFuncEntriesResult,     // #Functions found or #entries required
    [in] DtapiHwFunc* pHwFuncs         // Hardware-function descriptions
);
```

Parameters

NumFuncEntries

Specifies the size, in number of **DtapiHwFunc** entries, of the caller-supplied *pHwFuncs* array.

NumFuncEntriesResult

Output parameter that receives the number of hardware functions found and described in *pHwFuncs*. The value of this parameter can be greater than *NumFuncEntries* (when **DtapiPciScan** returns **DTAPI_E_BUFFER_TOO_SMALL**).

pHwFuncs

Points to a caller-supplied array of **DtapiHwFunc** entries to receive descriptions of the hardware functions.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Scan has completed successfully and the <i>pHwFuncs</i> array was large enough to contain all function descriptions.
DTAPI_E_BUF_TOO_SMALL	The number of function-description entries in <i>pHwFuncs</i> is too small. The number of entries required is returned in <i>NumFuncEntriesResult</i> .

Remarks

This function still exists for backward compatibility. Use **DtDeviceScan** in new software.

DtapiPciScan scans the PCI bus in the current system and lists the hardware functions available on the detected DTA-xxx PCI cards. Each card may contain multiple hardware functions.

This function may have to be called two times. The first time, *NumFuncEntries* should be set to a best-guess maximum value. If the result status is **DTAPI_E_BUF_TOO_SMALL**, the application should free the current array of **DtapiHwFunc** entries, allocate a new array with the number of entries returned in *NumFuncEntriesResult*, and call **DtapiPciScan** again.

::DtapiResult2Str

Convert DTAPI_RESULT value to a string.

```
const char* ::DtapiResult2Str(  
    [in] DTAPI_RESULT  DtapiResult    // DTAPI_RESULT value to be converted  
);
```

Parameters

DtapiResult

DTAPI_RESULT value to be converted to a string.

Result

Remarks

For ease of use, this function doesn't return a DTAPI_RESULT but returns the string directly.

DtDevice**DtDevice::AttachToSerial**

Attach device object to the device hardware, based on the serial number of the device.

```
DTAPI_RESULT DtDevice::AttachToSerial (
    [in] __int64 Serial,        // Serial number
);
```

Parameters

Serial

Serial number of the DekTec device to which the device object is to be attached.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Device object has been attached successfully to the hardware
DTAPI_E_ATTACHED	Device object is already attached to device hardware
DTAPI_E_NO_DEVICE	No DekTec devices found (at all)
DTAPI_E_NO_SUCH_DEVICE	The device with the specified serial number could not be found
DTAPI_E_DRIVER_INCOMP	Version of device driver is incompatible with the DTAPI version, device driver needs to be upgraded

Remarks

AttachToSerial is non-intrusive. No initialisation actions are performed.

DtDevice::AttachToSlot

Attach device object to a PCI Bus device, based on PCI-bus number and slot number.

```
DTAPI_RESULT DtDevice::AttachToSlot (
    [in] int   PciBusNumber,    // PCI-bus number
    [in] int   SlotNumber      // PCI-slot number
);
```

Parameters

PciBusNumber, SlotNumber

PCI-bus number and slot number of the DekTec device to which the device object is to be attached.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Device object has been attached successfully to the hardware
DTAPI_E_ATTACHED	Device object is already attached to a PCI card
DTAPI_E_NO_DTA_CARD	No DekTec PCI cards found (at all)
DTAPI_E_NO_SUCH_DEVICE	No DekTec DTA-1xx PCI card found in the specified slot, or the slot is empty
DTAPI_E_DRIVER_INCOMP	Version of device driver is incompatible with the DTAPI version, device driver needs to be upgraded

Remarks

AttachToSlot is non-intrusive. No initialisation actions are performed.

This method cannot be applied to USB devices. Use **AttachToSerial** or **AttachToType** instead.

DtDevice::AttachToType

Attach device object to the device hardware, based on the type number of the device.

```
DTAPI_RESULT DtDevice::AttachToType (
    [in] int   TypeNumber,          // Which device to look for
    [in] int   DeviceNo=0          // Relative device number
);
```

Parameters

TypeNumber

Integer value representing the type number of the device to which the device object is to be attached. The integer corresponds to the number in the hardware's type string, e.g. 100 for the DTA-100 or 225 for DTU-225.

DeviceNo

If the system contains multiple devices of the same type, this number distinguishes between the various devices. *DeviceNo* of the first device is 0, the next device 1, and so on.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Device object has been attached successfully to the hardware
DTAPI_E_ATTACHED	Device object is already attached to device hardware
DTAPI_E_INTERNAL	Internal DTAPI error
DTAPI_E_NO_DEVICE	No DekTec devices found (at all)
DTAPI_E_NO_SUCH_DEVICE	No device with type <i>Typenumber</i> is found in this system, or the number of devices of this type is less-or-equal than <i>DeviceNo</i>
DTAPI_E_DRIVER_INCOMP	Version of device driver is incompatible with the DTAPI version, device driver needs to be upgraded

Remarks

AttachToType is non-intrusive. No initialisation actions are performed.

DtDevice::Detach

Detach device object from device hardware.

```
DTAPI_RESULT DtDevice::Detach (  
    void  
);
```

Parameters

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Device object has been detached successfully from the device hardware
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware, so it cannot be detached

Remarks

DtDevice::GetDescriptor

Get device descriptor.

```
DTAPI_RESULT DtDevice::GetDescriptor (
    [out] DtDeviceDesc& DvcDesc    // Device descriptor
);
```

Parameters

DvcDesc

Output parameter that receives the device descriptor.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Version numbers have been retrieved successfully
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware

Remarks

DtDevice::GetDeviceDriverVersion

Get device-driver version information. Two overloaded variants of this function are provided. The first function prototype, which is defined for backward compatibility, returns the major and minor version number. The second version also provides bug fix and build number.

```
DTAPI_RESULT DtDevice::GetDeviceDriverVersion (
    [out] int&  DriverVersionMajor,
    [out] int&  DriverVersionMinor
);

DTAPI_RESULT DtDevice::GetDeviceDriverVersion (
    [out] int&  DriverVersionMajor,
    [out] int&  DriverVersionMinor,
    [out] int&  DriverVersionBugFix,
    [out] int&  DriverVersionBuild
);
```

Parameters

DriverVersionMajor

Major version number of the device driver used to access the device hardware. This number is incremented for major functional upgrades of the device driver.

DriverVersionMinor

The minor version number is incremented for small functional increments of the device driver.

DriverVersionBugFix

The bug-fix version number is incremented when a bug in the device driver is fixed, without functional enhancements.

DriverVersionBuild

Build number.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Version numbers have been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware

Remarks

This function cannot be used to obtain hardware versioning information. Use **VpdRead** instead.

DtDevice::GetFirmwareVersion

Get version number of the firmware loaded on the device.

```
DTAPI_RESULT DtDevice::GetFirmwareVersion (  
    [out] int& FirmwareVersion  
);
```

Parameters

FirmwareVersion

Single number that identifies the version of the FPGA- and/or embedded software on the device.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Firmware version has been retrieved successfully.
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver.
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware.

Remarks

DtDevice::GetIoConfig

Get the channel type of a physical port.

```
DTAPI_RESULT DtDevice::GetIoConfig (
    [in] int    Port,           // Physical port
    [out] int&  IoConfig        // DTAPI_IOCONFIG_XXX
);
```

Parameters

Port

Physical port number.

IoConfig

Port configuration, see **DtDevice::SetIoConfig**

If the port doesn't support I/O configuration then *Config* is set to **DTAPI_IOCONFIG_NOTSUP**.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	I/O configuration has been read successfully
DTAPI_E_NO_SUCH_PORT	Invalid port number for this device
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware

Remarks

DtDevice::GetRefClkCnt

Get a sample of the reference-clock counter on the device.

```
DTAPI_RESULT DtDevice::GetRefClkCnt (
    [out] int& RefClkCnt
);
```

Parameters

RefClkCnt

Sample of the 32-bit reference clock counter.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Sample has been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware
DTAPI_E_NOT_SUPPORTED	The device does not support retrieval of a sample of the reference-clock counter

Remarks

This method is supported on the following devices:

Device Type Number	Firmware Version
DTA-120	V4 or higher
DTA-140	V1 or higher

DtDevice::GetUsbSpeed

Get the speed (e.g. full or high speed) of the USB bus.

```
DTAPI_RESULT DtDevice::GetUsbSpeed (
    [out] int&   UsbSpeed
);
```

Parameters

UsbSpeed

Current speed of the USB bus the device is connected to.

Value	Meaning
DTAPI_USB_FULL_SPEED	USB bus operates at full speed (max. 12Mbps)
DTAPI_USB_HIGH_SPEED	USB bus operates at high speed (max. 480Mbps)

Result

DTAPI_RESULT	Meaning
DTAPI_OK	USB speed has been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware
DTAPI_E_NOT_SUPPORTED	The device does not support the getting of the USB speed

Remarks

Use this method to determine if the USB device is connected to a USB bus operating in full or high speed mode. A USB bus operating at full speed usually indicates that the DTU-2XX device is connected to a USB-1 bus⁴. High speed is only supported by USB-2 buses.

NOTE: full speed limits the maximum input/output bit-rate supported by the DTU-2XX devices to 8Mbps. To be able to use the DTU-2XX for bit-rates higher than 8Mbps a USB-2 bus operating at high speed should be used.

This method is only supported by the DTU-2XX devices.

⁴ USB-2 buses can operate in full-speed mode for backward compatibility reasons (support for USB-1 devices)

DtDevice::HwFuncScan

Scan hardware functions hosted by this device.

```
DTAPI_RESULT DtDevice::HwFuncScan (
    [in] int    NumEntries,           // #Function entries in pHwFuncs
    [out] int&  NumEntriesResult,    // #Functions found or #entries required
    [out] DtHwFuncDesc* pHwFuncs    // Hardware-function descriptions
);
```

Parameters

NumEntries

Specifies the size, in number of **DtHwFuncDesc** entries, of the caller-supplied *pHwFuncs* array.

NumEntriesResult

Output parameter that receives the number of hardware functions found and described in *pHwFuncs*. The value of this parameter can be greater than *NumEntries* (when **DtapiHwFuncScan** returns **DTAPI_E_BUFFER_TOO_SMALL**).

pHwFuncs

Pointer to a caller-supplied array of **DtHwFuncDesc** entries to receive the hardware-function descriptors.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Scan has completed successfully and the <i>pHwFuncs</i> array was large enough to contain all function descriptions.
DTAPI_E_BUF_TOO_SMALL	The number of function-description entries in <i>pHwFuncs</i> is too small. The number of entries required is returned in <i>NumEntriesResult</i> .

Remarks

This function is the equivalent of **::DtapiHwFuncScan** for a single device.

DtDevice::HwFuncScan function may have to be called twice. The first time, *NumEntries* should be set to a best-guess maximum value. If the result status is **DTAPI_E_BUF_TOO_SMALL**, the application should free the current array of **DtHwFuncDesc** entries, allocate a new array with the number of entries returned in *NumEntriesResult*, and call **DtapiHwFuncScan** again.

DtDevice::I2CRead

Read data from the I2C bus.

```
DTAPI_RESULT DtDevice::I2CRead (
    [in] int    DvcAddr,           // I2C device address
    [out] char* pBuffer,          // Buffer for receiving the data
    [in] int    NumBytesToRead,   // #Bytes to read
);
```

Parameters

DvcAddr

Device address of the targeted I2C device.

The I2C device address consists out of 1 transfer direction bit + 7 address bits. This method ignores the transfer bit (LSB) and only used the 7 address bits. Valid values for the device address are: 0x00h-0xFF

pBuffer

Pointer to a buffer for receiving the I2C bytes.

The buffer must be caller-allocated and have a size of at least NumBytesToRead.

NumBytesToRead

Number of bytes to read.

Maximum allowed number of bytes to read is 512.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Sample has been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware
DTAPI_E_INVALID_BUF	Invalid buffer pointer provided (pBuffer=NULL)
DTAPI_E_INVALID_SIZE	Invalid number of bytes to read specified (i.e. >512 bytes)
DTAPI_E_NOT_SUPPORTED	The device does not support reading of the onboard I2C bus

Remarks

The I2CRead method is intended for direct low-level access to the onboard I2C resources.

DtDevice::I2CWrite

Write data to the I2C bus.

```
DTAPI_RESULT DtDevice::I2CWrite (
    [in] int    DvcAddr,           // I2C device address
    [in] char*  pBuffer,          // Buffer with bytes to write
    [in] int    NumBytesToWrite, // #Bytes to write
);
```

Parameters

DvcAddr

Device address of the targeted I2C device

The I2C device address consists out of 1 transfer direction bit + 7 address bits. This method ignores the transfer bit (LSB) and only used the 7 address bits. Valid values for the device address are: 0x00h-0xFF

pBuffer

Pointer to a buffer with the bytes to write.

The buffer must have a size of at least NumBytesToWrite.

NumBytesToWrite

Number of bytes to write.

Maximum allowed number of bytes to write is 512.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Sample has been retrieved successfully.
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver.
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware.
DTAPI_E_INVALID_BUF	Invalid buffer pointer provided (pBuffer=NULL)
DTAPI_E_INVALID_SIZE	Invalid number of bytes to write specified (i.e. >512 bytes)
DTAPI_E_NOT_SUPPORTED	The device does not support writing to the onboard I2C bus.

Remarks

The I2C Write method is intended for direct low-level access to the onboard I2C resources.

DtDevice::LedControl

Take direct control of the device's general-status LED, or let the hardware drive the LED.

```
DTAPI_RESULT DtDevice::LedControl (
    [in] int    LedControl      // DTAPI_LED_XXX
);
```

Parameters

LedControl

Controls status of the LED.

Value	Meaning
DTAPI_LED_HARDWARE	Hardware drives the LED (default after power up)
DTAPI_LED_OFF	LED is forced to off-state
DTAPI_LED_GREEN	LED is forced to green-state
DTAPI_LED_RED	LED is forced to red-state
DTAPI_LED_YELLOW	LED is forced to yellow-state

Result

DTAPI_RESULT	Meaning
DTAPI_OK	LED setting has been accepted
DTAPI_E_INVALID_MODE	The specified LED-control value is invalid
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware
DTAPI_E_NOT_SUPPORTED	The device does not have a general-status LED

Remarks

When a device object is detached from the device hardware, all direct-control settings are released (LED control is reset to **DTAPI_LED_HARDWARE**).

The DTA-120, DTA-122 and DTA-140 each have a single LED, which can be controlled by either this method (**DtDevice::LedControl**) or by **DtInpChannel::LedControl**. If both methods are applied in parallel, **DtDevice::LedControl** has precedence over **DtInpChannel::LedControl**.

DtDevice::SetIoConfig

Configure a physical port:

- Setting the direction of bi-directional ASI/SDI ports (DTA-145, DTA-160)
- For selecting whether an ASI/SDI output is an independent output, or a double buffered copy of another port (DTA-105)

```
DTAPI_RESULT DtDevice::SetIoConfig (
    [in] int    Port,           // Physical port
    [in] int    IoConfig       // DTAPI_IOCONFIG_XXX
);
```

Parameters

Port

Physical port number.

IoConfig

Port configuration

Value	Meaning
DTAPI_IOCONFIG_INPUT	Configure port as an ASI/SDI input
DTAPI_IOCONFIG_OUTPUT	Configure port as an ASI/SDI output DTA-105, Port #2: configure port as an independent output
DTAPI_IOCONFIG_DBLBUF	Configure port as a doubly-buffered copy of another port DTA-105, Port #2: make signal on Port #2 a doubly buffered copy of Port #2

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Channel type has been set successfully
DTAPI_E_ATTACHED	Cannot change I/O configuration when a channel object is attached to this port
DTAPI_E_INVALID_MODE	Invalid setting for I/O configuration
DTAPI_E_NO_SUCH_PORT	Invalid port number for this device
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware
DTAPI_E_NOT_SUPPORTED	The specified port doesn't permit I/O configuration

Remarks

The I/O configuration of a port can only be set if the device is attached to a **DtDevice** object. The channel may not be attached to a channel object.

The new I/O configuration is persisted in the registry. The current I/O configuration of a port can be read back with **DtDevice::GetIoConfig**.

DtDevice::VpdDelete

Delete a Vital-Product Data (VPD) item from the VPD read/write section in the serial EEPROM on the device.

```
DTAPI_RESULT DtDevice::VpdDelete (  
    [in] char*   pKeyword      // Keyword identifying VPD item, e.g. "Y0"  
);
```

Parameters

pKeyword

Null-terminated character string identifying the VPD item to be deleted.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	VPD item has been deleted successfully
DTAPI_E_EEPROM_READ	A read operation from the serial EEPROM did not succeed
DTAPI_E_EEPROM_WRITE	The write operation to the serial EEPROM did not succeed
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware
DTAPI_E_NOT_FOUND	The VPD item could not be found
DTAPI_E_READ_ONLY	An attempt was made to delete a read-only VPD item

Remarks

If a VPD item with the specified keyword already exists, that item is overwritten, unless it is a read-only item. In the latter case, **DTAPI_E_READ_ONLY** is returned.

The size of the VPD read/write segment is 256 bytes. Writing to the serial EEPROM is a relatively slow operation.

DtDevice::VpdRead

Read a Vital-Product Data (VPD) item from the EEPROM on the device.

```
DTAPI_RESULT DtOutpChannel::VpdRead (  
    [in] char*    pKeyword,        // Keyword identifying VPD item, e.g. "SN"  
    [out] char*    pVpdItem        // String read from EEPROM  
);
```

Parameters

pKeyword

Null-terminated character string identifying the VPD item to be read. The keyword must consist of either two characters, or should be the special string "VPDID".

The table below lists standard keywords supported by DekTec devices. Next to these standard keywords, custom VPD keywords can be created with **VpdWrite**.

Value	Meaning
"VPDID"	Pseudo value to retrieve the VPD ID String, e.g. "DTA-100 DVB-ASI-C Output 0..150 Mbps".
"CL"	Customer ID
"EC"	Engineering Change level. Identifies the hardware revision level of the device, e.g. "Rev 1".
"MN"	Manufacture ID DekTec-internal code identifying the manufacturer of the hardware.
"PD"	Production Date, e.g. "2003.07"
"PN"	Part Number, e.g. "DTU-225"
"SN"	Serial Number E.g. "4225266001".
"XT"	Crystal stability E.g. "5ppm@25C;15ppm", which means a frequency stability of ± 5 ppm at room temperature and a stability of ± 15 ppm over the full temperature range and including aging.
"BO"	Bit-rate Offset e.g. 1.52ppm, Offset that is used to fine tuning the bit-rate value that is return by the device

pVpdItem

String retrieved from the EEPROM. The character array must be allocated before calling **VpdRead**. The **DTAPI** limits the maximum length of a VPD item to 63 characters, so a 64-char array suffices.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	VPD item has been read successfully
DTAPI_E_EEPROM_FORMAT	The data format in the serial EEPROM is not VPD compliant
DTAPI_E_EEPROM_READ	A read operation from the serial EEPROM did not succeed
DTAPI_E_KEYWORD	The keyword is neither two characters, nor "VPDID"
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware
DTAPI_E_NOT_FOUND	The VPD item could not be found

Remarks

If one of the standard keywords ("CL", "EC", ...) has been specified and the method returns `DTAPI_E_NOT_FOUND`, the serial EEPROM has been tampered.

DtDevice::VpdWrite

Write a Vital-Product Data (VPD) item to the VPD read/write section in the serial EEPROM on the device.

```
DTAPI_RESULT DtDevice::VpdWrite (
    [in] char*   pKeyword,        // Keyword identifying VPD item, e.g. "Y1"
    [in] char*   pVpdItem        // String to be written to the EEPROM
);
```

Parameters

pKeyword

Null-terminated character string identifying the VPD item to be written. The keyword must consist of two characters (the "VPDID" item cannot be written).

pVpdItem

String to be written to the EEPROM. The maximum size of a VPD item is 63 characters.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	VPD item has been written successfully
DTAPI_E_EEPROM_FULL	The serial EEPROM has not enough free space available for writing the new VPD item
DTAPI_E_EEPROM_READ	A read operation from the serial EEPROM did not succeed
DTAPI_E_EEPROM_WRITE	The write operation to the serial EEPROM did not succeed for another reason
DTAPI_E_KEYWORD	The keyword does not consist of two characters
DTAPI_E_NOT_ATTACHED	Device object is not attached to device hardware
DTAPI_E_READ_ONLY	An attempt was made to overwrite a read-only VPD item
DTAPI_E_TOO_LONG	The length of the VPD item is too long (>63 characters)

Remarks

If a VPD item with the specified keyword already exists, that item is overwritten, unless it is a read-only item. In the latter case, **DTAPI_E_READ_ONLY** is returned.

For system-specific use, The VPD specification in the *PCI Local Bus Specification Rev 2.2* recommends keywords of the form "Yx", with the second character one of '0' ... '9', 'B' ... 'Z'. Keyword "YA" is defined as the *system-asset identifier* provided by the system owner. Keywords of the form "Vx" are reserved for use by DekTec.

The size of the VPD read/write segment is 256 bytes. Write operations to the serial EEPROM are relatively slow.

DtInpChannel**DtInpChannel**

Class representing an input channel for receiving the following formats:

- MPEG-2 Transport Stream over ASI, SPI or IP
- Serial Digital Interface (SDI)

```
class DtInpChannel;
```

Derived Classes*SdiInpChannel*

Class representing an input channel for receiving SDI.
MPEG-2 TS-specific methods applied to this class will fail.

TsInpChannel

Class representing an input channel for receiving an MPEG-2 Transport Stream.
SDI-specific methods applied to this class will fail.

DtInpChannel::Attach

Attach the input-channel object to a hardware function hosted by a device.

```
DTAPI_RESULT DtInpChannel::Attach (
    [in] DtDevice*  pDtDvc,           // Device object
    [in] int        InpIndex=0,       // Relative index
    [in] bool       ProbeOnly=false   // Just check whether channel is in use
);
```

Parameters

pDtDvc

Pointer to the device object that represents a DekTec device. The device object must have been attached to the device hardware using method **DtDevice::AttachToSerial**, **DtDevice::AttachToSlot** or **DtDevice::AttachToType**.

InpIndex

If the device hosts multiple hardware input functions, this index specifies to which hardware function, and therefore to which physical input, the channel object is to be attached. The first hardware input function has an input-index value of 0, the next function 1, and so on.

ProbeOnly

Probe whether the channel is in use, but do not actually attach.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Channel object has been attached successfully to the hardware function
DTAPI_E_ATTACHED	Channel object is already attached to a hardware function
DTAPI_E_DEVICE	Pointer <i>pDtDvc</i> is not valid or the device object is not attached to a hardware device
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_IN_USE	Another channel object is already attached to the hardware function
DTAPI_E_NO_DT_INPUT	No inputs on this device
DTAPI_E_NO_SUCH_INPUT	<i>InpIndex</i> refers to a non-existing hardware function
DTAPI_E_OUT_OF_MEM	TS-over-IP: Receive FIFO cannot be allocated

Remarks

On the DTA-160, this method cannot be used to attach to the TS-over-IP Ethernet port. Please use **DtInpChannel::AttachToPort** instead.

Next to establishing the link between input-channel object and hardware function, **Attach** also performs the following initialisation actions:

- The contents of the Receive FIFO are cleared
- Receive-control state is reset to **DTAPI_RXCTRL_IDLE**

- Loop-back mode is reset to `DTAPI_NOLOOPBACK`
- All *latched* status flags are cleared (refer to `ClearFlags`)

DtInpChannel::AttachToPort

Attach the input-channel object to a specific physical port.

```
DTAPI_RESULT DtInpChannel::AttachToPort (
    [in] DtDevice*  pDtDvc,           // Device object
    [in] int        Port,             // Port number
    [in] bool       ProbeOnly=false   // Just check whether channel is in use
);
```

Parameters

pDtDvc

Pointer to the device object that represents a DekTec device. The device object must have been attached to the device hardware.

Port

Physical port number. The channel object is attached to this port. The port number of the top-most port is 1, except on the DTA-160, on which the top-most Ethernet port is port #4. Please refer to Section 4 for an overview of port numbers.

ProbeOnly

Probe whether the channel is in use, but do not actually attach.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Channel object has been attached successfully to the port
DTAPI_E_ATTACHED	Channel object is already attached
DTAPI_E_DEVICE	Pointer <i>pDtDvc</i> is not valid or the device object is not attached to a hardware device
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_IN_USE	Another channel object is already attached to this port
DTAPI_E_NO_DT_INPUT	<i>Port</i> is not an input
DTAPI_E_NO_SUCH_PORT	<i>Port</i> refers to a non-existing port number
DTAPI_E_OUT_OF_MEM	TS-over-IP: Receive FIFO cannot be allocated

Remarks

AttachToPort performs the same initialisation actions as **DtInpChannel::Attach**.

DtInpChannel::ClearFifo

Clear contents of the Receive FIFO and set receive-control state to **DTAPI_RXCTRL_IDLE**. Clear the receive-FIFO-overflow flag (**DTAPI_RX_FIFO_OVF**).

```
DTAPI_RESULT DtInpChannel::ClearFifo (  
    void  
);
```

Parameters

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Receive FIFO has been cleared
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The effects of **ClearFifo** are equivalent to **Reset(DTAPI_FIFO_RESET)**.

DtInpChannel::ClearFlags

Clear *latched* status flag(s).

```
DTAPI_RESULT DtInpChannel::ClearFlags (
    [int] int    Latched          // Latched status flags to be cleared
);
```

Parameters

Latched

Latched status flag(s) to be cleared. Multiple flags can be cleared with one method call by OR-ing the bit positions to be cleared. The following flags are latched and can be cleared:

Value	Meaning
DTAPI_RX_FIFO_OVF	See GetFlags
DTAPI_RX_SYNC_ERR	" "
DTAPI_RX_RATE_OVF	" "
DTAPI_RX_TARGET_ERR	" "

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Flag(s) have been cleared successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

Some status flags that are queried with **GetFlags** are not latched and therefore cannot be cleared.

The latched status flags are automatically reset after the DTAPI-calls: **DtDevice::AttachToSerial**, **DtDevice::AttachToSlot**, **DtDevice::AttachToType** and after **Reset**.

DtInpChannel::Detach

Detach input-channel object from hardware function and free associated resources.

```
DTAPI_RESULT DtInpChannel::Detach (  
    [in] int DetachMode        // How to detach  
);
```

Parameters

DetachMode

Specifies how the channel object is detached from the hardware function.

If *DetachMode* is 0, the object is detached without further action. Other modes are defined below.

Value	Meaning
DTAPI_INSTANT_DETACH	Clear the contents of the Receive FIFO and set the receive-control state to DTAPI_RXCTRL_IDLE

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Channel object has been detached successfully from the hardware function
DTAPI_E_INVALID_FLAGS	An invalid detach flag was specified
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function, so it cannot be detached

Remarks

DtInpChannel::Equalise

Obsolete.

Turn DVB-ASI cable equalisation on or off (DTA-120 Rev 1 only).

```
DTAPI_RESULT DtInpChannel::Equalise (
    [in] int EqualiserSetting // Equaliser on/off
);
```

Parameters

EqualiserSetting

This parameter specifies the equaliser setting, according to the following table.

Value	Meaning
DTAPI_EQUALISER_OFF	No equalisation; for short cable lengths (<10m)
DTAPI_EQUALISER_ON	Provide 100m of coaxial cable equalisation

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Equaliser setting has been successfully applied
DTAPI_E_INVALID_MODE	The specified equalisation setting is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The DTA-120 Rev 2, DTA-124, DTA-140 and DTU-255 contain auto-equalisation circuitry. For these devices, **Equalise** returns **DTAPI_E_INVALID_MODE**.

DtInpChannel::GetFifoLoad

Get the current load of the input-channel's Receive FIFO.

```
DTAPI_RESULT DtInpChannel::GetFifoLoad (
    [out] int& FifoLoad          // Load of Receive FIFO
);
```

Parameters

FifoLoad

Number of bytes in the Receive FIFO.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	FIFO load has been retrieved successfully.
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function.

Remarks

The value retrieved with this method call approximates the load of the Receive FIFO. Some additional data bytes may be buffered on the device.

If a transfer is in progress and/or the device receives data, then every call to **GetFifoLoad** may return a different value.

DtInpChannel::GetFlags

Get status flags for the input channel.

```
DTAPI_RESULT DtInpChannel::GetFlags (
    [out] int&  Flags,           // Status flags
    [out] int&  Latched         // Latched status flags
);
```

Parameters

Flags

Output parameter that is set to the current values of the input-channel status flags. Each status flag is represented by one bit. Multiple status flags can be true simultaneously. If none of the status flags is true, *Status* is set to zero.

Value	Meaning
DTAPI_RX_FIFO_OVF	A Receive-FIFO overflow condition has occurred. The data in the Receive FIFO could not be transferred fast enough to a system buffer.
DTAPI_RX_SYNC_ERR	A synchronisation error has occurred in the packet-synchronisation logic of the input channel. Note: A synchronisation error cannot occur in packet mode DTAPI_RXMODE_RAW.
DTAPI_RX_RATE_OVF	Data is entering the system faster than the input channel can process the data (applies to DTA-122 only). When the input rate on the DTA-122 remains below 150 Mbit/s, this error cannot occur.
DTAPI_RX_TARGET_ERR	The target adapter signals a fault (DTA-122 only)

Latched

Output parameter that is set to the latched values of the status flags: On a '0' to '1' transition of a status flag, the corresponding bit in *Latched* is set to '1'. The bit remains set until cleared explicitly by one of the following DTAPI-calls: **ClearFlags**, **DtDevice::AttachToSerial**, **DtDevice::AttachToSlot**, **DtDevice::AttachToType** or **Reset**.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Status flags have been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

DtInpChannel::GetMaxFifoSize

Get the maximum size of the Receive FIFO on the device.

```
DTAPI_RESULT DtInpChannel::GetMaxFifoSize (
    [out] int& MaxFifoSize    // Maximum size of FIFO in bytes
);
```

Parameters

MaxFifoSize

Maximum size of the receive FIFO in number of bytes.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Maximum size of receive FIFO has been read successfully
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The DTA-120, DTA-122, DTA-124, DTA-140, DTU-225 and DTU-234 support a maximum Receive-FIFO size of 8.388.608 bytes. Later revisions of these devices support at least this size, but potentially the maximum FIFO size may become larger.

At the present the FIFO size for an input channel cannot be modified. However future DekTec devices might support the setting the maximum Receive-FIFO size.

DtInpChannel::GetReceiveByteCount

This function is not available yet in the current version of the DTAPI

Get a sample from the free running 32-bit received-number-of-bytes counter.

```
DTAPI_RESULT DtInpChannel::GetReceiveByteCount (
    [out] int& ByteCount    // Sample of #received-bytes counter
);
```

Parameters

ByteCount

Sample of the 32-bit received number of bytes counter. For SDI 10-bit words are counted.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	A valid sample of the counter was returned
DTAPI_E_NOT_SUPPORTED	The received number of bytes counter is not supported by the current input channel
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

This method is supported on the following devices:

Device Type Number	Firmware Version
DTA-120	V4 or higher
DTA-122	V4 or higher
DTA-124	V0 or higher
DTA-140	V0 or higher
DTU-225	V2 or higher

DtInpChannel::GetRxControl

Get the current value of the channel's receive-control state.

```
DTAPI_RESULT DtInpChannel::GetRxControl (
    [out] int& RxControl    // Receive-control state
);
```

Parameters

RxControl

This parameter is set to the current value of the receive-control state: **DTAPI_RXCTRL_IDLE** or **DTAPI_RXCTRL_RCV**.

Refer to **SetRxControl** for a description of these states.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Receive-control state has been successfully retrieved
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

Transport Streams

In receive modes **DTAPI_RXMODE_ST188**, **DTAPI_RXMODE_STMP2** and **DTAPI_RXMODE_ST204**, receive-control state is synchronised to packet boundaries. For example, if **SetRxControl** is used to change the control state from **DTAPI_RXCTRL_RCV** to **DTAPI_RXCTRL_IDLE**, and **GetRxControl** is called immediately thereafter, then **DTAPI_RXCTRL_RCV** may be returned. Only when a new packet enters the Receive FIFO, the value returned by **GetRxControl** becomes **DTAPI_RXCTRL_IDLE**.

SDI

Receive-control state is synchronised to the vertical sync.

In receive mode **DTAPI_RXMODE_STRAW**, method **GetRxControl** always returns the receive-control state set by **SetRxControl**.

DtInpChannel::GetStatistics

Get statistics information from the input channel.

```
DTAPI_RESULT DtInpChannel::GetStatistics (
    [out] int& ViolCount    // Code-violation counter (ASI inputs only)
);
```

Parameters

ViolCount

Total number of DVB-ASI code violations since power-up of a DVB-ASI input channel (DTA-120, DTA-124, DTA-140, DTU-225). A code violation is a bit error that leads to an illegal 8B/10B code (the line code used by DVB-ASI). Bit errors may be caused by poor cable quality, or by an input cable that is too long.

The value of this counter is updated about 20 times per second. The counter is only incremented and never reset. When the largest positive int value ($2^{31}-1$) has been reached, the counter wraps around to the largest negative int value (-2^{31}).

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Statistics information has been read successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

Connecting or disconnecting the cable to/from a DVB-ASI input channel may cause a massive amount of code violations. This is "normal" behaviour, caused by the locking process of the DVB-ASI input circuitry.

DtInpChannel::GetStatus

Get status information from the input channel. If a device does not support a certain feature, the corresponding status variable is set to **DTAPI_NOT_SUPPORTED**.

```
DTAPI_RESULT DtInpChannel::GetStatus (
[out] int& PacketSize,      // Packet size
[out] int& NumInv,          // #Invalid bytes per packet (DTA-122)
[out] int& ClkDet,          // Clock- or carrier detected
[out] int& AsiLock,         // DVB-ASI PLL locked (ASI inputs)
[out] int& RateOk,          // Input rate above minimum
[out] int& AsiInv           // Input-invert status (ASI inputs)
);
```

Parameters

PacketSize

Size of incoming MPEG-2 transport packets.

Value	Meaning
DTAPI_PCKSIZE_188	188-byte packets at the Transport-Stream input
DTAPI_PCKSIZE_204	204-byte packets at the Transport-Stream input
DTAPI_PCKSIZE_INV	No MPEG-2 compliant packets found at the Transport-Stream input

NumInv

Defined for DVB-SPI input channels (DTA-122) only: Number of "invalid" bytes (DVALID input signal is '0') per packet.

Value	Meaning
DTAPI_NUMINV_NONE	No invalid bytes
DTAPI_NUMINV_16	16 invalid bytes per packet
DTAPI_NUMINV_OTHER	Other number of invalid bytes per packet
DTAPI_NOT_SUPPORTED	Device does not support this parameter (not DTA-122)

ClkDet

For DVB-SPI input channels, this output parameter indicates whether a receive clock of sufficient frequency is detected at the SPI input;

For DVB-ASI and SDI input channels, this output parameter acts as a *Carrier Detect* signal.

Value	Meaning
DTAPI_CLKDET_OK	DVB-SPI : Receive clock detected DVB-ASI, SDI : Carrier detected TS-over-IP : IP traffic detected in the last second
DTAPI_CLKDET_FAIL	DVB-SPI : No receive clock detected, or receive-clock rate is too low DVB-ASI : No carrier detected TS-over-IP : No IP traffic in the last second

AsiLock

Defined for DVB-ASI input channels only: Output parameter that indicates whether the DVB-ASI clock signal can be recovered reliably.

Value	Meaning
DTAPI_ASI_INLOCK	PLL is locked to the incoming DVB-ASI input signal
DTAPI_ASI_NOLOCK	Clock signal cannot be recovered from the input signal
DTAPI_NOT_SUPPORTED	Hardware function does not support this parameter

RateOk

Defined for DVB-ASI input channels only: Output parameter that indicates whether the transport rate at the DVB-ASI input is sufficiently high for further processing. When this parameter is set to **DTAPI_INPRATE_LOW**, the most likely cause is an "empty" DVB-ASI signal (stuffing symbols only).

Value	Meaning
DTAPI_INPRATE_OK	The DVB-ASI input rate is sufficient
DTAPI_INPRATE_LOW	The DVB-ASI input rate is too low (<900 bps)
DTAPI_NOT_SUPPORTED	Hardware function does not support this parameter

AsiInv

Defined for DVB-ASI input channels only: This parameter indicates whether the input circuitry is currently inverting the DVB-ASI input signal. This is most useful when polarity control has been set to **DTAPI_POLARITY_AUTO**; In the other polarity-control settings, *AsiInv* just echoes the value of parameter *PolarityControl* in the call to **PolarityControl**.

Value	Meaning
DTAPI_ASIINV_NORMAL	Polarity of DVB-ASI input signal is normal (not inverted)
DTAPI_ASIINV_INVERT	Polarity of DVB-ASI signal is inverted
DTAPI_NOT_SUPPORTED	Device does not support this parameter (DTA-122)

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Status information has been read successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

If no input signal is applied, output parameter *PacketSize* is set to **DTAPI_PCKSIZE_INV**.

DtInpChannel::GetTargetId

Get the target-adapter identifier (DTA-122 only).

```
DTAPI_RESULT DtInpChannel::GetTargetId (
    [out] int& Present,          // Target adapter present?
    [out] int& TargetId         // Target-adapter identifier
);
```

Parameters

Present

Output parameter that indicates whether a target adapter has been detected.

Value	Meaning
DTAPI_NO_CONNECTION	Nothing is connected to the input connector of the DTA-122.
DTAPI_DVB_SPI_SOURCE	A standard DVB-SPI source is connected to the DTA-122.
DTAPI_TARGET_PRESENT	A target adapter is present.
DTAPI_TARGET_UNKNOWN	The device is busy assessing the situation on the input connector.

TargetId

Output parameter that is set to an integer value that uniquely identifies the target adapter. Please refer to the DTA-122 documentation for a list of available target adapters.

TargetId is assigned a value only if *Present* is **DTAPI_TARGET_PRESENT**.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Target ID has been retrieved successfully.
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver.
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function.
DTAPI_E_NOT_SUPPORTED	The input channel does not support target adapters (DVB-ASI input channels: DTA-120/140, DTU-225).

Remarks

The DTA-122 does not recognize the DTA-102 as a standard DVB-SPI source (*Present* is set to **DTAPI_NO_CONNECTION**), unless the ground pins on the DVB-SPI cable are connected together. This is due to the target-adapter detection circuitry.

DtInpChannel::GetTsRateBps

Get an estimate of the input Transport-Stream rate.

```
DTAPI_RESULT DtInpChannel::GetTsRateBps (
    [out] int& TsRate           // Transport-Stream rate in bps
);
```

Parameters

TsRate

Estimate of the current Transport-Stream rate, expressed in bits per second. This rate does not take into account 'extra' bytes beyond the 188 MPEG-2 defined bytes.

If the channel's receive mode is **DTAPI_RXMODE_STRAW**, the value returned by this method is equal to the raw input bit rate, this is the rate at which valid data enters the device.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Transport-Stream rate has been read successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

In all receive modes except **DTAPI_RXMODE_STRAW**, this method strictly applies the definition of Transport-Stream rate in the MPEG-2 Systems specification. This rate is based on 188-byte packets. If the packet size is not 188 bytes, a conversion factor is used.

Example: When 204-byte packets enter the system, the raw input rate is divided by 204/188.

DtInpChannel::LedControl

Take direct control of input-status LED, or let hardware drive the LED.

```
DTAPI_RESULT DtInpChannel::LedControl (
    [in] int    LedControl          // DTAPI_LED_XXX
);
```

Parameters

LedControl

Controls the LED.

Value	Meaning
DTAPI_LED_HARDWARE	Hardware drives the LED (default after power-up)
DTAPI_LED_OFF	LED is forced to off-state
DTAPI_LED_GREEN	LED is forced to green-state
DTAPI_LED_RED	LED is forced to red-state
DTAPI_LED_YELLOW	LED is forced to yellow-state

Result

DTAPI_RESULT	Meaning
DTAPI_OK	LED setting has been accepted
DTAPI_E_INVALID_MODE	The specified LED-control value is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

Detaching the input channel releases any direct-control setting that might have been applied to the LEDs (LED control is reset to **DTAPI_LED_HARDWARE**).

The DTA-120, DTA-122 and DTA-140 each have a single LED, which can be controlled by either this method (**DtDevice::LedControl**) or by **DtInpChannel::LedControl**. If both methods are used at the same time, then **DtDevice::LedControl** takes precedence over **DtInpChannel::LedControl**.

DtInpChannel::PolarityControl

Control the automatic polarity-detection circuitry of a DVB-ASI Transport-Stream input channel.

```
DTAPI_RESULT DtInpChannel::PolarityControl (
    [in] int  PolarityControl // Polarity-control setting
);
```

Parameters

PolarityControl

This parameter controls inversion of the DVB-ASI signal.

Value	Meaning
DTAPI_POLARITY_AUTO	Automatically detect and correct polarity of DVB-ASI signal
DTAPI_POLARITY_NORMAL	'Normal' operation: do not invert DVB-ASI signal
DTAPI_POLARITY_INVERT	Invert DVB-ASI signal

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Polarity setting has been accepted
DTAPI_E_INVALID_MODE	The specified polarity-control value is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The DVB-ASI signal is sensitive to signal polarity. Without corrective measures, an inverted DVB-ASI signal – which may be caused by e.g. inverting distribution amplifiers – may be decoded incorrectly by a standard DVB-ASI receiver.

Automatic detection of DVB-ASI signal polarity (setting **DTAPI_POLARITY_AUTO**) can be successfully applied only when it is known a priori that the input signal is DVB/MPEG-2 compliant. For non MPEG-2 applications, *PolarityControl* should be set to **DTAPI_POLARITY_NORMAL**, or the input signal may be distorted badly due to periodic inversion.

DtInpChannel::Read

Read data bytes from the input channel.

```
DTAPI_RESULT DtInpChannel::Read (
    [in] char*   pBuffer,           // Buffer to store data
    [in] int     NumBytesToRead    // #Bytes to be read (must be multiple of 4!)
);
```

Parameters

pBuffer

Pointer to the buffer into which the data bytes from the input channel will be written.
The pointer must be aligned to a 4-byte address boundary.

NumBytesToRead

Transfer size: Number of bytes to be read from the input channel.
The value of this parameter must be positive and a multiple of four.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Read operation has been completed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_BUF	The buffer is not aligned to a 32-bit word boundary
DTAPI_E_INVALID_SIZE	The specified transfer size is negative or not a multiple of four
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

Read returns when *NumBytesToRead* bytes have been transferred into the buffer. The transfer size may be greater than the initial number of bytes available in the Receive FIFO. The thread executing **Read** sleeps until sufficient data has entered the Receive FIFO to complete the transfer.

If the transfer size is greater than the initial Receive-FIFO load, and either the input signal disappears or Receive Control is **DTAPI_RXCTRL_IDLE**, then the Read call may sleep for an indefinite period of time (the thread 'hangs'). To avoid this situation, it is recommended to first check the FIFO load and then read an amount of data less than or equal to the FIFO load.

For PCI-card devices, the DTAPI applies DMA to directly transfer the data from PCI card to the buffer in user space. Hereto, the device driver locks the physical pages of the buffer in memory. A scatter/gather list is created and the DMA transfer is initiated.

For USB-devices, the data has to pass the USB device-driver stack. DMA may, or may not, be applied.

DtInpChannel::ReadDirect

This function is for diagnostics purposes only, please use method `Read` for 'ordinary' reading
 PCI cards only: Read data bytes from the input channel using "direct" read-cycles over the PCI bus.

```
DTAPI_RESULT DtInpChannel::ReadDirect (
    [out] char*  pBuffer,           // Buffer to store data
    [in] int    NumBytesToRead,    // Number of bytes to be read
    [out] int&   NumBytesRead      // Number of bytes read
);
```

Parameters

pBuffer

Pointer to the buffer to which the data from the input channel will be written.
 The pointer must be aligned to a 32-bit word boundary.

NumBytesToRead

Size of the read buffer, and as such the maximum number of bytes read from the input channel. The buffer size must be positive and a multiple of four.

NumBytesRead

Number of bytes actually read. *NumBytesRead* is always a multiple of four.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Read operation has been completed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_BUF	The buffer is not aligned to a 32-bit word boundary
DTAPI_E_INVALID_SIZE	The specified transfer size is negative or not a multiple of four
DTAPI_E_FIFO_EMPTY	The operation timed out because the Receive FIFO is empty
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

Method `ReadDirect` is very similar to `Read`. For PCI cards (DTA-1XX), `ReadDirect` is considerably slower than `Read` because the processor reads the data bytes directly via the PCI bus, instead of using DMA. `ReadDirect` should only be used for special purposes, e.g. for PCI-card validation.

The data buffer can be any buffer in user space that is aligned to a 4-byte boundary.

`ReadDirect` returns when the read buffer is full or when the Receive FIFO is empty.

Unlike `Read`, this function cannot block forever. If the read operation takes too long, error code `DTAPI_E_FIFO_EMPTY` is returned.

For USB devices, `ReadDirect` is equivalent to `Read`.

DtInpChannel::Reset

Reset input channel.

```
DTAPI_RESULT DtInpChannel::Reset (
    [in] int ResetMode
);
```

Parameters

ResetMode

Specifies which part of the hardware and software stack is reset. The following values are defined (values cannot be OR-ed together):

Value	Meaning
DTAPI_FIFO_RESET	Reset (clear) the Receive FIFO: • Data transfers are halted instantaneously • All data pending in the Receive FIFO is discarded • Receive-control state is reset to DTAPI_RXCTRL_IDLE • Receive-FIFO overflow flag is cleared
DTAPI_FULL_RESET	Full input-channel reset: • All actions for DTAPI_FIFO_RESET, plus: • Synchronisation-error flag (DTAPI_RX_SYNC_ERR) is cleared • State machines in the device hardware are reset

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Input channel has been reset
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_MODE	The value specified for <i>ResetMode</i> is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

An input-channel reset operation does not affect the following settings:

- Receive mode and insert-time-stamp flag (refer to **DtInpChannel::SetRxMode**)
- Polarity control of DVB-ASI inputs (refer to **DtInpChannel::PolarityControl**)

DtInpChannel::SetIpPars

Set parameters for the reception of a Transport Stream over IP and start listening.

```
DTAPI_RESULT DtInpChannel::SetIpPars (
    [in] DtTsIpPars* pTsIpPars    // TS-over-IP parameters
);
```

Parameters

SetIpPars

New parameter set to be applied. Please refer to the **DtTsIpPars** page for a description of the parameters.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	TS-over-IP parameters have been applied successfully
DTAPI_E_IN_USE	Parameters cannot be changed because the channel is busy. The receive-control state should be switched back to idle first
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

SetIpPars should be called at least once after attaching to the hardware but before setting the receive-control state to **DTAPI_RXCTRL_RCV**.

After the initial call to **SetIpPars**, parameters can be changed again, but only when the receive-control state is **DTAPI_RXCTRL_IDLE**.

When the destination IP address is a multicast IP address, and *pTsIpPars->m_MulticastJoin* is 1, the DTAPI automatically joins the multicast group upon the first invocation of **SetIpPars**. When this method is called again, membership of the old multicast group is first dropped and a new multicast group is joined if required.

DtInpChannel::SetLoopBackMode

This function is for diagnostics purposes only

Set the input-channel's loop-back mode.

```
DTAPI_RESULT DtInpChannel::SetLoopBackMode (
    [in] int Mode           // Loop-back mode
);
```

Parameters

Mode

Loop-back mode according to the table below.

Value	Meaning
DTAPI_NO_LOOPBACK	Normal operation, no loop-back of data
DTAPI_LOOPBACK_MODE	Loop-back mode. The physical input circuitry is disconnected from the Receive FIFO. Data can be written directly into the Receive FIFO using WriteLoopBackData .

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Loop-back state has been successfully changed
DTAPI_E_INVALID_MODE	The value specified for loop-back mode is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

After **Attach** and after **Reset**, loop-back mode is turned off.

DtInpChannel::SetPower

DTA-122 only. Turn on/off power for a target adapter attached to the DTA-122.

```
DTAPI_RESULT DtInpChannel::SetPower (
    [in] int    Power          // Power state
);
```

Parameters

Power

Power state according to the table below.

Value	Meaning
DTAPI_POWER_OFF	No power is applied. The 25-pin sub-D connector is compatible with DVB-SPI
DTAPI_POWER_ON	Apply power (+5V) to pin 12 and 25 of the 25-pin sub-D connector

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Power state has been changed successfully
DTAPI_E_INVALID_MODE	The specified power-mode value is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_NOT_SUPPORTED	The device does not support a power connection for target adapters

Remarks

After **Attach** and after **Reset**, power is turned off.

DtInpChannel::SetRxControl

Set the channel's receive-control state.

```
DTAPI_RESULT DtInpChannel::SetRxControl (
    [in] int RxControl    // Receive-control state
);
```

Parameters

RxControl

New receive-control state according to the table below.

Value	Meaning
DTAPI_RXCTRL_IDLE	The input stream input is "disconnected" from the Receive FIFO: Incoming transport packets are not stored in the Receive FIFO.
DTAPI_RXCTRL_RCV	Normal operation. Incoming transport packets are stored in the Receive FIFO.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Receive-control state has been changed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_MODE	The specified receive-control state is invalid or incompatible with the attached hardware function
DTAPI_E_NO_IP_PARS	For TS-over-IP channels: receive-control state cannot be set to DTAPI_RXCTRL_RCV because TS-over-IP parameters have not been specified
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

If the receive-control state is set to **DTAPI_RXCTRL_RCV**, but the application does not read any data from the Receive FIFO, then the Receive FIFO will quickly overflow.

Receive-control state is initialised to **DTAPI_RXCTRL_IDLE** by calling **DtDevice::AttachToSlot**, **DtDevice::AttachToType**, **DtDevice::AttachtoSerial** or **DtInpChannel::Reset**.

DtInpChannel::SetRxMode

Set the channel's receive mode. The receive mode determines the (hardware) processing that is applied to the incoming stream.

```
DTAPI_RESULT DtInpChannel::SetRxMode (
    [in] int    RxMode           // Receive mode
);
```

Parameters

RxMode

Receive mode according to the table below.

Value	Meaning
DTAPI_RXMODE_ST188	<i>188-byte mode</i> Always store 188-byte packets in the Receive FIFO. When the input contains 204-byte packets, the 16 trailing bytes are dropped. Input data without 188- or 204-byte packet structure is dropped.
DTAPI_RXMODE_ST204	<i>204-byte mode</i> Always store 204-byte packets in the Receive FIFO. When the input contains 188-byte packets, 16 zero bytes are appended. Input data without 188- or 204-byte packet structure is dropped.
DTAPI_RXMODE_STMP2	<i>MPEG-2 mode</i> Store 188- or 204-byte packets in the Receive FIFO without modification. Input data without 188- or 204-byte packet structure is dropped.
DTAPI_RXMODE_STRAW	<i>Raw mode</i> No notion of packets. All incoming valid data bytes are stored in the Receive FIFO. NOTE: This mode is incompatible with DTAPI_POLARITY_AUTO! Please refer to the Remarks section.
DTAPI_RXMODE_STTRP	<i>Transparent mode</i> All incoming data bytes are stored in the Receive FIFO. The data is aligned to packet boundaries if valid packets are detected. This format includes a trailer that contains information about the detected packet size, sync status and valid data bytes within the packet.

The following mode is valid for TS-over-IP reception only.

Value	Meaning
DTAPI_RXMODE_IPRAW	<i>Raw IP mode</i> Store unprocessed IP packets in the buffer. If error correction is requested, store FEC streams too. NOTE: each IP packet returned by a call the DtInpChannel::Read will be preceded by a DtIpRawHeader structure.

The receive mode can be optionally combined (OR-ed) with the following flag:

Value	Meaning
DTAPI_RX_TIMESTAMP	<i>Time-stamped mode</i> Insert a 32-bit time stamp before each packet. The time stamp is a sample of the system clock counter on the device.

The **DTAPI_RX_TIMESTAMP** flag cannot be specified in raw mode (**DTAPI_RXMODE_STRAW**).

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Receive mode has been changed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_MODE	The specified receive mode is invalid or incompatible with the input channel
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

In receive mode **DTAPI_RXMODE_STRAW** ("raw" mode), the input channel does not care about the packet structure of the incoming data: All data bytes are stored in the input buffer. For DVB-ASI input channels, raw mode can only work reliably if *polarity control* is set to **DTAPI_POLARITY_NORMAL** or **DTAPI_POLARITY_INVERT**. If polarity control is set to **DTAPI_POLARITY_AUTO**, disaster may be the result: Automatic polarity detection assumes that the input has a valid packet structure. If such a structure cannot be found, the device tries again with the input signal inverted. In raw mode, such inversion may occur periodically and severely corrupt the input data!

For the DTA-122 DVB-SPI Input Adapter, packet synchronisation in modes **DTAPI_RXMODE_ST188** and **DTAPI_RXMODE_ST204** is based on the PSYNC signal, not on the value of the first byte of the packet: The value of DATA at a PSYNC pulse is stored in the input buffer, even if the value is not 0x47.

The receive mode **DTAPI_RXMODE_STTRP** the incoming data is always transmitted in 204 byte packets followed by a 4 byte trailer that contains information about the data bytes. The device actively looks for packet boundaries and if found will synchronise to the packet boundary in such a way that the first byte will be a packet sync (0x47). To do this stuff bytes might be send to achieve packet sync. The trailer indicates if synchronisation has been achieved, the sync byte is valid and how much of the 204 bytes transmitted are actual packet data bytes. Note that when 188 byte packets are received the device will also send 204 bytes were 188 bytes will be valid.

The trailer contains the following bytes:

Field	Meaning
Status Byte	1 Byte field were: Bit 7..3 is always 01100b Bit 2 is '1' if the data bytes are aligned with packet sync Bit 1 is '1' if the sync byte is not equal to 0x47 Bit 0 is '1' if 204 byte packets are detected is '0' if 188 byte packets are detected
Sequence Number	2 Byte field containing a packet sequence number that is increased on every incoming packet
Valid Count	1 Byte field indicating the number of bytes that are actual data bytes

Time stamps are stored in Little Endian format: the first byte contains the least significant 8 bits, the fourth byte the most significant 8 bits. On 32-bit platforms that use the Little-Endian convention (a.o. Intel IA-32), the time stamp can be read by code like:

```
unsigned int TimeStamp = *(unsigned int*) PtrInCharBuffer;
```

DtInpChannel::WriteLoopBackData

This function is for diagnostics purposes only

Write data bytes to the channel's Receive FIFO via the loop-back channel. The loop-back mode must be **DTAPI_LOOPBACK_MODE**.

```
DTAPI_RESULT DtInpChannel::WriteLoopBackData (
    [in] char*  pBuffer,          // Data written to Receive FIFO
    [in] int    NumBytesToWrite // Number of bytes to be written
);
```

Parameters

pBuffer

Pointer to the buffer that will supply the data written to the Receive FIFO. The size of the buffer must be at least *NumBytesToWrite*.

NumBytesToWrite

Number of bytes to be written to the Receive FIFO. *NumBytesToWrite* must be positive.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Write operation has been completed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_SIZE	The specified transfer size is negative
DTAPI_E_NO_LOOPBACK	Channel is not in loop-back mode
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

This method can be used for writing a Receive-FIFO memory test.

Data can only be written to the Receive FIFO if the loop-back mode is **DTAPI_LOOPBACK_MODE**. This method does not check for Receive-FIFO overflow.

DtOutpChannel**DtOutpChannel**

Class representing an output channel for transmitting the following formats:

- MPEG-2 Transport Stream over ASI, SPI or IP
- Serial Digital Interface (SDI)

```
class DtOutpChannel;
```

Derived Classes*SdiOutpChannel*

Class representing an output channel for transmitting SDI.
MPEG-2 TS-specific methods applied to this class will fail.

TsOutpChannel

Class representing an output channel for transmitting an MPEG-2 Transport Stream.
SDI-specific methods applied to this class will fail.

DtOutpChannel::Attach

Attach the output-channel object to a hardware function hosted by a particular device. Two overloaded variants are defined.

- The first variant attaches to a hardware function of a **DtDevice** object and should be used in new software.
- The second – obsolete – function prototype, which is defined for backward compatibility with DTAPI version 1.3.x.xxx, attaches the output-channel object to a hardware function of a **PCI Card** object.

```
DTAPI_RESULT DtOutpChannel::Attach (
    [in] DtDevice*  pDtDevice,          // Object representing device
    [in] int  OutpIndex=0,              // Output index
    [in] bool  ProbeOnly=false         // Just check whether channel is in use
);

DTAPI_RESULT DtOutpChannel::Attach (      // Obsolete
    [in] PciCard*  pPciCard,           // Object representing PCI card
    [in] int  OutpIndex=0              // Output index on PCI card
);
```

Parameters

pDtDvc

Pointer to object that represents a DekTec device. The **DtDevice** object must be attached to the device hardware.

OutpIndex

If the device has multiple hardware output functions, this number specifies to which hardware function, and therefore to which physical output, the channel object is attached. The *OutpIndex* of the first hardware output function is 0.

ProbeOnly

Probe whether the channel is in use, but do not actually attach.

pPciCard

Pointer to object that represents a PCI-card. The **PciCard** object must be attached to the device hardware.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Channel object has been attached successfully to the hardware function
DTAPI_E_ATTACHED	The channel object is already attached a hardware function
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_IN_USE	The output is already in use by another channel object
DTAPI_E_NO_DT_OUTPUT	The device does not contain hardware output functions
DTAPI_E_NO_SUCH_OUTPUT	<i>OutpIndex</i> refers to a non-existing output
DTAPI_E_DEVICE DTAPI_E_PCICARD	The DtDevice pointer is not valid or the DtDevice object is not attached to the device hardware

Remarks

On the DTA-160, this method cannot be used to attach to the TS-over-IP Ethernet port. Please use **DtOutpChannel::AttachToPort** instead.

Next to establishing the link with the hardware, **Attach** also performs the following initialisation:

- The contents of the Transmit FIFO are cleared.
- The size of the Transmit FIFO is set to the maximum value.
- Transmit-control state is reset to **DTAPI_TXCTRL_IDLE**.
- Loop-back mode is reset to "no loop-back".
- All *latched* status flags are cleared (refer to **ClearFlags**).
- Enable LVDS output signals (DTA-102 only).

For Transport-Stream outputs, **Attach** does not reset the Transport-Stream rate to zero. This allows switching between applications that use the same output, without causing a dip in the Transport-Stream.

DtOutpChannel::AttachToPort

Attach the output-channel object to a specific physical port.

```
DTAPI_RESULT DtOutpChannel::AttachToPort (
    [in] DtDevice*  pDtDvc,           // Device object
    [in] int        Port,             // Port number
    [in] bool       ProbeOnly=false   // Just check whether channel is in use
);
```

Parameters

pDtDvc

Pointer to the device object that represents a DekTec device. The device object must have been attached to the device hardware.

Port

Physical port number. The channel object is attached to this port. The port number of the top-most port is 1, except on the DTA-160, on which the top-most Ethernet port is port #4. Please refer to Section 4 for an overview of port numbers.

ProbeOnly

Probe whether the channel is in use, but do not actually attach.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Channel object has been attached successfully to the port
DTAPI_E_ATTACHED	Channel object is already attached
DTAPI_E_DEVICE	Pointer <i>pDtDvc</i> is not valid or the device object is not attached to a hardware device
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_IN_USE	Another channel object is already attached to this port
DTAPI_E_NO_DT_OUTPUT	<i>Port</i> is not an output
DTAPI_E_NO_SUCH_PORT	<i>Port</i> refers to a non-existing port number
DTAPI_E_OUT_OF_MEM	TS-over-IP: Receive FIFO cannot be allocated

Remarks

AttachToPort performs the same initialisation actions as **DtOutpChannel::Attach**.

DtOutpChannel::ClearFifo

Clear contents of the Transmit FIFO and set transmit-control state to `DTAPI_TXCTRL_IDLE`. Furthermore, clear the output channel's status flags: the transmit-FIFO-underflow flag (`DTAPI_TX_FIFO_UFL`) and the transmit-synchronisation-error flag (`DTAPI_TX_SYNC_ERR`).

```
DTAPI_RESULT DtOutpChannel::ClearFifo (  
    void  
);
```

Parameters

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Transmit FIFO has been cleared
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The effects of `ClearFifo` are equivalent to `Reset(DTAPI_FIFO_RESET)`.

`ClearFifo` does not reset the last part of the hardware pipeline, so that packets are not truncated (unless transmit mode is `DTAPI_TXMODE_RAW`.)

Method `ClearFifo` is available on all DekTec output cards, except on DTA-100 firmware versions prior to V3, and on DTA-102 firmware versions prior to V5. On these elder PCI cards, `ClearFifo` has the same effect as `Reset(DTAPI_FULL_RESET)`.

DtOutpChannel::ClearFlags

Clear *latched* status flag(s).

```
DTAPI_RESULT DtOutpChannel::ClearFlags (
    [int] int    Latched          // Latched status flags to be cleared
);
```

Parameters

Latched

Latched status flag(s) to be cleared. Multiple flags can be cleared with one function call by OR-ing the bit positions to be cleared. The following flags are latched and can be cleared:

Value	Meaning
DTAPI_TX_FIFO_UFL	See GetFlags
DTAPI_TX_READBACK_ERR	„ „
DTAPI_TX_SYNC_ERR	„ „
DTAPI_TX_TARGET_ERR	„ „

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Flag(s) have been successfully cleared
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

Some status flags that are queried with **GetFlags** are not latched and therefore cannot be cleared.

The latched status flags are also automatically reset after calling **AttachToSlot**, **AttachToType** and after **Reset**. A call to **ClearFifo** clears **DTAPI_TX_FIFO_UFL** and **DTAPI_TX_SYNC_ERR**.

DtOutputChannel::Detach

Detach output channel object from a hardware function. Free resources allocated for the output channel, such as DMA buffers.

```
DTAPI_RESULT DtOutputChannel::Detach (
    [in] int DetachMode          // How to detach
);
```

Parameters

DetachMode

Specifies how the channel object should detach from the hardware function. If *DetachMode* is 0, the object is detached without further action. A number of flags listed below are defined to detach from the hardware function in a specific way. The flags can be OR-ed together to their combine behaviour, with some exceptions as listed in the table.

Value	Meaning
DTAPI_INSTANT_DETACH	Clear the contents of the Transmit FIFO and detach without waiting until pending data in the FIFO has been transmitted. This flag may not be combined with DTAPI_WAIT_UNTIL_SENT.
DTAPI_TRISTATE_OUTPUT	Put all output signals in tri-state (DTA-102 only).
DTAPI_WAIT_UNTIL_SENT	Sleep until all pending data in the Transmit FIFO has been transmitted. If this flag is combined with other flags, the wait is executed before the action associated with the other flags. This flag may not be combined with DTAPI_INSTANT_DETACH.
DTAPI_ZERO_OUTPUT	Set the Transport-Stream rate to zero. On the DTA-100/140, only stuffing symbols are generated. On the DTA-102, all output signals are reset to constant '0'.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Channel object has been detached successfully from the hardware function
DTAPI_E_INVALID_FLAGS	An invalid combination of detach flags was specified
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function, so it cannot be detached

Remarks

If the client does not specify **DTAPI_ZERO_OUTPUT** and/or **DTAPI_TRISTATE_OUTPUT**, and packet stuffing is turned on, the output channel keeps transmitting null packets after detaching.

If the hardware does not support tri-stating of the output, specifying **DTAPI_TRISTATE_OUTPUT** acts as a no-op. No error result is generated.

In certain circumstances **Detach** may "hang", e.g. if **DTAPI_WAIT_UNTIL_SENT** is specified while the FIFO still contains data and the transmit-control state is **DTAPI_TXCTRL_IDLE**.

DtOutpChannel::GetExtClkFreq

Get an estimate of the frequency of the external clock (DTA-102 only).

```
DTAPI_RESULT DtOutpChannel::GetTsRateBpsExtClk (  
    [out] int& ExtClkFreq    // Measurement of external-clock frequency  
);
```

Parameters

TsRate

Output parameter that is set to a measurement of the frequency of the signal applied to the external-clock input. For an accurate estimate, the external clock signal must be present and stable for at least one second.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	External-clock frequency has been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The frequency returned is a *byte* rate that must be multiplied by 8 or $8 \cdot 204/188$ to obtain the corresponding Transport-Stream rate.

DtOutpChannel::GetFifoLoad

Get the current load of the channel's Transmit FIFO.

```
DTAPI_RESULT DtOutpChannel::GetFifoLoad (
    [out] int& FifoLoad          // Load of Transmit FIFO
);
```

Parameters

FifoLoad

Number of bytes in the Transmit FIFO.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	FIFO load has been retrieved successfully
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The value retrieved with this method call *approximates* the load of the Transmit FIFO. The actual number of bytes buffered on the card may be up to 2180 bytes higher. *FifoLoad* cannot be less than the Transmit-FIFO load.

If a Data transfer is in progress and/or the transmit-control state is **DTAPI_TXCTRL_SEND**, then every call to **GetFifoLoad** may return a different value.

DtOutpChannel::GetFifoSize

Get the current size of the channel's Transmit FIFO.

```
DTAPI_RESULT DtOutpChannel::GetFifoSize (
    [out] int& FifoSize           // Size of Transmit FIFO in bytes
);
```

Parameters

FifoSize

Size of the Transmit FIFO in number of bytes.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	FIFO size has been retrieved successfully
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

DtOutpChannel::GetFlags

Get status flags of the output channel.

```
DTAPI_RESULT DtOutpChannel::GetFlags (
    [out] int&  Status,          // Status flags
    [out] int&  Latched         // Latched status flags
);
```

Parameters

Status

Output parameter that is set to a value describing the current status of the output channel. Each status flag is represented by one bit. Multiple status flags can be set at the same time. If none of the status flags is asserted, *Status* is set to zero.

Value	Meaning
DTAPI_TX_DMA_PENDING DTAPI_TX_PENDING	A data transfer is pending. No latched version of this flag is available.
DTAPI_TX_FIFO_UFL	A Transmit-FIFO underflow condition has occurred. Underflow detection is available in all transmit modes, including modes with null-packet stuffing switched on.
DTAPI_TX_SYNC_ERR	A Transmit-FIFO synchronisation-error has occurred. The size of one or more packets in the output mode does not match the Transmit Mode. This status flag is not used in Transmit Mode DTAPI_TXMODE_RAW.
DTAPI_TX_READBACK_ERR	The hardware has detected that an output pin is forced to an erroneous signal level, e.g. because of a short-circuit (DTA-102 only).
DTAPI_TX_TARGET_ERR	The target adapter signals a fault (DTA-102 only).

Latched

Output parameter that *latches* the value of the status flags: If a status flag has become '1', even for a very short moment, the corresponding bit in *Latched* is set to '1'. The bit remains set until cleared explicitly by **ClearFlag**, or implicitly by one of the following DTAPI-calls: **ClearFifo**, **AttachToSlot**, **AttachToType** or **Reset**.

The following flag is not latched: **DTAPI_TX_PENDING**; the corresponding bit position is always set to '0'.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Status flags have been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The **DTAPI_TX_PENDING** flag can be used to poll whether a Data transfer is complete. However, it is discouraged to create a polling loop that waits until DMA is done. Instead, use **DmaWaitForDone** to suspend the thread until DMA is done.

DtOutpChannel::GetMaxFifoSize

Get the maximum size of the Transmit FIFO on the device.

```
DTAPI_RESULT DtOutpChannel::GetMaxFifoSize (
    [out] int& MaxFifoSize    // Maximum size of FIFO in bytes
);
```

Parameters

MaxFifoSize

Maximum size of the Transmit FIFO in number of bytes.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Maximum size of Transmit FIFO has been read successfully
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The actual size of the Transmit-FIFO is adjustable in software with **SetFifoSize**.

The DTA-100, DTA-102 and DTA-140 support a maximum Transmit-FIFO size of 8.388.608 bytes. Later revisions of these boards support at least this size, but potentially the maximum FIFO size may become larger.

DtOutpChannel::GetModControl

Get modulation-control parameters for Transport-Stream output channels with a built-in modulator.

```
DTAPI_RESULT DtOutpChannel::GetModControl (
    [out] int&  ModType,           // Modulation type
    [out] int&  CodeRate,         // Convolutional rate
    [out] int&  ParXtra1,         // Extra parameter #1
    [out] int&  ParXtra2         // Extra parameter #2
);
```

Parameters

ModType

Output parameter that is set to the modulation type. See **SetModControl1** for a list of applicable values.

CodeRate

Output parameter that is set to the convolutional rate used for puncturing the modulated signal. See **SetModControl1** for a list of applicable values.

ParXtra1, ParXtra2

Extra modulation parameters that may be used for new modulator cards, or when a new firmware revision is introduced with extra functionality.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	The modulation parameters have been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_NOT_SUPPORTED	The device does not include a modulator

Remarks

DtOutpChannel::GetRfControl

Get up-converter parameters for devices with on-board RF up-converter.

```
DTAPI_RESULT DtOutpChannel::GetRfControl (
    [in] __int64& RfRate,      // RF frequency in Hz
    [out] int& LockStatus      // Lock status of RF PLL
);
```

Parameters

RfRate

Output parameters that is set to the current carrier frequency as programmed into the RF up-converter, expressed Hertz.

The RF frequency returned in *RfRate* may be different from the frequency programmed with **SetRfControl** because of rounding to the RF step size (see **SetRfControl** for a list of step sizes).

LockStatus

Output parameter that is set to the status of the RF up-converter PLL.

Value	Meaning
DTAPI_RFPLL_LOCK	The RF PLL is in lock. This is the normal state in which the up-converter is generating a stable carrier frequency.
DTAPI_RFPLL_NO_LOCK	The RF PLL is out of lock. This status may occur temporarily when the RF rate is changed to another value. The no-lock status may occur permanently when the RF rate has been programmed (outside DTAPI) to a value that is incompatible with the output channel.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	The up-converter parameters have been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_NOT_SUPPORTED	The device does not have an RF up-converter

Remarks

DtOutpChannel::GetTargetId

Get the target-adapter identifier (DTA-102 only).

```
DTAPI_RESULT DtOutpChannel::GetTargetId (
    [out] int& Present,           // Target adapter present?
    [out] int& TargetId          // Target-adapter identifier
);
```

Parameters

Present

Output parameter that indicates whether a target adapter has been detected.

Value	Meaning
DTAPI_NO_CONNECTION	Nothing is connected to the output connector of the DTA-102
DTAPI_DVB_SPI_SINK	A standard DVB-SPI sink is connected to the DTA-102
DTAPI_TARGET_PRESENT	A target adapter is present
DTAPI_TARGET_UNKNOWN	The system is busy assessing the situation on the output connector

TargetId

Output parameter that is set to an integer value that uniquely identifies the target adapter. Refer to the DTA-102 documentation for a list of available target adapters.

A value is assigned to *TargetId* only if *Present* is **DTAPI_TARGET_PRESENT**.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Data has been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_NOT_SUPPORTED	The output channel does not support target adapters

Remarks

DtOutpChannel::GetTransmitByteCount

This function is not available yet in the current version of the DTAPI

Get a sample from the free running 32-bit transmitted-number-of-bytes counter.

```
DTAPI_RESULT DtOutpChannel::GetTransmitByteCount (
    [out] int& ByteCount    // Sample of #transmitted-bytes counter
);
```

Parameters

ByteCount

Sample of the 32-bit transmitted-number-of-bytes counter.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	A valid sample of the counter was returned
DTAPI_E_NOT_SUPPORTED	The counter is not supported by the current output channel
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

This method is supported on the following devices:

Device Type Number	Firmware Version
DTA-100	V7 or higher
DTA-102	V9 or higher
DTA-107	V1 or higher
DTA-140	V3 or higher
DTU-205	V1 or higher

DtOutpChannel::GetTsRateBps

Get the channel's current clock-generator mode and Transport-Stream rate.

```
DTAPI_RESULT DtOutpChannel::GetTsRateBps (
    [out] int& ClockGenMode,    // Internal or external clock
    [out] int& TsRate          // Transport-Stream rate in bps
);
```

Parameters

ClockGenMode

Output parameter that is set to the current clock-generator mode.

Value	Meaning
DTAPI_TXCLOCK_INTERNAL	Internal clock generator
DTAPI_TXCLOCK_EXTERNAL	External clock input (DTA-102 only)

TsRate

Output parameter that is set to the current Transport-Stream rate expressed in bits per second. If an external clock generator is used, *TsRate* is set to a measurement of the Transport-Stream rate corresponding to the signal on the external clock input. For an accurate estimate, the external clock signal must be present and stable for also at least one second.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Transport-Stream rate has been read successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The same remarks regarding Transport-Stream rate vs. transmit-clock rate apply as listed for **SetTsRateBps**.

DtOutpChannel::GetTxControl

Get the channel's current transmit-control state.

```
DTAPI_RESULT DtOutpChannel::GetTxControl (
    [out] int& TxControl    // Transmit-control state
);
```

Parameters

TxControl

Refer to **SetTxControl** for a description of transmit-control states.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Transmit-control state has been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

DtOutpChannel::GetTxMode

Get the channel's current transmit mode.

```
DTAPI_RESULT DtOutpChannel::GetTxMode (
    [out] int& TxMode,           // Transmit mode
    [out] int& StuffMode        // Null-packet stuffing on/off
);
```

Parameters

TxMode, StuffMode

Refer to **SetTxMode** for a description of transmit-control modes.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Transmit mode has been retrieved successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

DtOutpChannel::ReadLoopBackData

This function is for diagnostics purposes only

Read data bytes from the channel's Transmit FIFO via the loop-back channel. The loop-back mode must be `DTAPI_LOOPBACK_MODE`.

```
DTAPI_RESULT DtOutpChannel::ReadLoopBackData (
    [out] char*  pBuffer,          // Data read back from Transmit FIFO
    [in] int    NumBytesToRead    // Number of bytes to be read
);
```

Parameters

pBuffer

Pointer to the buffer that will receive the data read from the Transmit FIFO. The size of the buffer must be at least *NumBytesToRead*. The buffer pointer must be aligned to a 32-bit word boundary.

NumBytesToRead

Number of bytes to be read from the Transmit FIFO. *NumBytesToRead* must be positive and a multiple of four.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Read operation has been completed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_BUF	The buffer is not aligned to a 32-bit word boundary
DTAPI_E_INVALID_SIZE	The specified transfer size is negative or not a multiple of four
DTAPI_E_NO_LOOPBACK	Channel is not in loop-back mode
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_UNDERFLOW	Transmit FIFO contains less than <i>NumBytesToRead</i> bytes

Remarks

This method can be used for writing a Transmit-FIFO memory test.

Data from the Transmit FIFO can only be read back if the loop-back mode is `DTAPI_LOOPBACK_MODE`.

DtOutpChannel::Reset

Reset output channel.

```
DTAPI_RESULT DtOutpChannel::Reset (
    [in] int ResetMode
);
```

Parameters

ResetMode

Specifies which part of the hardware and software should be reset. The following values are defined (values cannot be OR-ed together):

Value	Meaning
DTAPI_FIFO_RESET	Reset the Transmit FIFO: • Data transfers and packet transmission are halted instantaneously. • All data pending in the Transmit FIFO is discarded. • Transmit-control state is reset to DTAPI_TXCTRL_IDLE. • Transmit-FIFO underflow flag is cleared.
DTAPI_FULL_RESET	Full reset: • All actions for DTAPI_FIFO_RESET, plus: • Transport-Stream rate is reset to zero (except for OFDM modulation on DTA-110T)

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Output channel has been reset
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_MODE	The specified value for <i>ResetMode</i> is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

A potential side-effect of calling **Reset** is that the packet currently being transmitted is truncated. For one packet, the number of bytes between two consecutive SYNC bytes is less than the packet size. To avoid such a truncation, **ClearFifo** may be used.

DtOutpChannel::SetFifoSize

Set the size the Transmit FIFO on the device to a specified value (**SetFifoSize**) or to the maximum value supported by the channel (**SetFifoSizeMax**).

```
DTAPI_RESULT DtOutpChannel::SetFifoSize (
    [in] int  FifoSize          // Size of FIFO in bytes
);
DTAPI_RESULT DtOutpChannel::SetFifoSizeMax ( void );
```

Parameters

FifoSize

Size of the Transmit FIFO in number of bytes.

FifoSize must be a multiple of 16 and may not exceed the maximum physical size of the Transmit FIFO (DTA-100, DTA-102 and DTA-140: 8.388.608 bytes).

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Size of the Transmit FIFO has been set successfully
DTAPI_E_IN_USE	The FIFO size cannot be changed because transmission-control state is DTAPI_TXCTRL_HOLD or DTAPI_TXCTRL_SEND
DTAPI_E_INVALID_SIZE	The specified FIFO size is negative, zero, not a multiple of 16 or greater than the maximum size
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The size of the Transmit FIFO determines the amount of packet data that is buffered on the device. It also determines the delay between transferring data to the device (with **DmaStartTransfer**) and transmission of that data.

A large value for *FifoSize* creates headroom for *jitter* in the supply of data. Jitter is caused by other processes that also need processor cycles, or e.g. by a disk that is seeking a new disk-head position. For applications in which the output data does not depend on any input – e.g. in a disk streamer application – it is recommended to set *FifoSize* to the largest possible value.

A small value for *FifoSize* enables a short end-to-end delay, which is suitable for data-only applications in which an occasional underflow of the Transmit FIFO may occur, e.g. IP gateway. For such applications, *FifoSize* may be set, for example, to twice the size of the DMA buffer. In the extreme the Transmit-FIFO size may even be set to 16 (the minimum), but then occasional null-packet stuffing is almost guaranteed.

For real-time applications that cannot tolerate buffer underflow – e.g. in a real-time encoder – a careful assessment must be made to find a suitable buffer size, involving both real-time analysis and a bit of experimentation.

DtOutpChannel::SetIpPars

Set parameters for the transmission of a Transport Stream over IP.

```
DTAPI_RESULT DtOutpChannel::SetIpPars (
    [in] DtTsIpPars* pTsIpPars    // TS-over-IP parameters
);
```

Parameters

SetIpPars

New parameter set to be applied. Please refer to the **DtTsIpPars** page for a description of the parameters.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	TS-over-IP parameters have been applied successfully
DTAPI_E_IN_USE	Parameters cannot be changed because the channel is busy. The transmit-control state should be switched back to idle first
DTAPI_E_INVALID_ARG	The value of one of the TS-over-IP parameters is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

SetIpPars should be called at least once after attaching to the hardware but before setting the transmit-control state to **DTAPI_TXCTRL_HOLD** or **DTAPI_TXCTRL_SEND**.

After the initial call to **SetIpPars**, parameters can be changed again, but only when the transmit-control state is **DTAPI_TXCTRL_IDLE**.

DtOutpChannel::SetLoopBackMode

This function is for diagnostics purposes only

Set the channel's loop-back mode.

```
DTAPI_RESULT DtOutpChannel::SetLoopBackMode (
    [in] int Mode           // New loop-back mode
);
```

Parameters

Mode

New loop-back mode according to the table below.

Value	Meaning
DTAPI_NO_LOOPBACK	Normal operation. No loop-back of data.
DTAPI_LOOPBACK_MODE	Loop-back mode. The physical output circuitry is disconnected from the Transmit FIFO. The data in the Transmit FIFO can be read using ReadLoopBackData .

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Loop-back state has been changed successfully
DTAPI_E_INVALID_MODE	The specified loop-back mode value is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

After **Attach** and after **Reset**, loop-back is turned off.

DtOutpChannel::SetModControl

Set modulation-control parameters for output channels with a built-in modulator.

```
DTAPI_RESULT DtOutpChannel::SetModControl (
    [in] int  ModType,           // Modulation type
    [in] int  CodeRate,         // Convolutional rate / Roll-off factor
    [in] int  ParXtral1,        // Extra parameter #1
    [in] int  ParXtral2        // Extra parameter #2
);
```

Parameters

ModType

Modulation type.

Device	Value	Meaning
DTA-107	DTAPI_MOD_QPSK	QPSK modulation
	DTAPI_MOD_BPSK	BPSK modulation
DTA-110	DTAPI_MOD_QAM4	4-QAM modulation
	DTAPI_MOD_QAM16	16-QAM modulation
	DTAPI_MOD_QAM32	32-QAM modulation
	DTAPI_MOD_QAM64	64-QAM modulation
	DTAPI_MOD_QAM128	128-QAM modulation
	DTAPI_MOD_QAM256	256-QAM modulation
DTA-110T	All modes of DTA-110, plus:	
	DTAPI_MOD_OFDM	OFDM modulation

CodeRate

DTA-107 and DTA-110T: Convolutional rate used for puncturing the modulated signal;

DTA-110: Channel-filter roll-off factor

Device	Value	Meaning
DTA-107	DTAPI_MOD_1_2	Code rate 1/2, as defined in DVB-S
	DTAPI_MOD_2_3	Code rate 2/3, as defined in DVB-S
	DTAPI_MOD_3_4	Code rate 3/4, as defined in DVB-S
	DTAPI_MOD_4_5	Code rate 4/5
	DTAPI_MOD_5_6	Code rate 5/6, as defined in DVB-S
	DTAPI_MOD_6_7	Code rate 6/7
	DTAPI_MOD_7_8	Code rate 7/8, as defined in DVB-S

DTA-110	DTAPI_MOD_ROLLOFF_13	13%, as defined in J.83 annex C
	DTAPI_MOD_ROLLOFF_15	15%, as defined in J.83 annex C (DVB-C)
DTA-110T	DTAPI_MOD_1_2	Code rate 1/2
	DTAPI_MOD_2_3	Code rate 2/3
	DTAPI_MOD_3_4	Code rate 3/4
	DTAPI_MOD_5_6	Code rate 5/6
	DTAPI_MOD_7_8	Code rate 7/8

ParXtral, *ParXtra2*

Extra modulation parameters that are reserved by DekTec for use with new modulator cards, or when a new firmware revision is introduced with extra functionality.

If these parameters are not used they should be set to 0.

ParXtral

Extra modulation parameter #1

For DTA-107 and DTA-110 this parameters is not used and should be set to 0.

For the DTA-110T, *ParXtral* is an OR of the following fields:

Bandwidth

Device	Value	Meaning
DTA-110T	DTAPI_MOD_OFDM_5MHZ	5 MHz
	DTAPI_MOD_OFDM_6MHZ	6 MHz
	DTAPI_MOD_OFDM_7MHZ	7 MHz
	DTAPI_MOD_OFDM_8MHZ	8 MHz

Constellation

Device	Value	Meaning
DTA-110T	DTAPI_MOD_OFDM_QPSK	QPSK
	DTAPI_MOD_OFDM_QAM16	16-QAM
	DTAPI_MOD_OFDM_QAM64	64-QAM

Guard Interval

Device	Value	Meaning
DTA-110T	DTAPI_MOD_OFDM_G_1_32	1/32
	DTAPI_MOD_OFDM_G_1_16	1/16
	DTAPI_MOD_OFDM_G_1_8	1/8
	DTAPI_MOD_OFDM_G_1_4	1/4

Interleaving

Device	Value	Meaning
DTA-110T	DTAPI_MOD_OFDM_INDEPTH	In-depth interleaver (2k, 4k)
	DTAPI_MOD_OFDM_NATIVE	Native interleaver

Transmission Mode

Device	Value	Meaning
DTA-110T	DTAPI_MOD_OFDM_2K	2k mode
	DTAPI_MOD_OFDM_4K	4k mode
	DTAPI_MOD_OFDM_8K	8k mode

DVB-H Signalling – Service Indication s48

Device	Value	Meaning
DTA-110T	0	Time slicing is not used
	DTAPI_MOD_OFDM_S48	At least one elementary stream uses Time Slicing

DVB-H Signalling – Service Indication s49

Device	Value	Meaning
DTA-110T	0	MPE-FEC is not used
	DTAPI_MOD_OFDM_S49	At least one elementary stream uses MPE-FEC

ParXtra2

Extra modulation parameter #2

For DTA-107 and DTA-110 this parameter is not used.

For the DTA-110T, *ParXtra2* is the 16-bit cell identifier (*cell_id*).

Result

DTAPI_RESULT	Meaning
DTAPI_OK	The modulation parameters have been set successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_BANDWIDTH	Invalid value for bandwidth field in <i>ParXtra1</i>
DTAPI_E_INVALID_CONSTEL	Invalid value for constellation field in <i>ParXtra1</i>
DTAPI_E_INVALID_GUARD	Invalid value for guard-interval field in <i>ParXtra1</i>
DTAPI_E_INVALID_INTERLVNG	Invalid value for interleaving field in <i>ParXtra1</i>
DTAPI_E_INVALID_MODE	Modulation type is incompatible with modulator
DTAPI_E_INVALID_RATE	Convolutional rate is incompatible with modulator

DTAPI_E_INVALID_ROLLOFF	Roll-off factor is incompatible with modulator
DTAPI_E_INVALID_TRANSMODE	Invalid value for transmission-mode field in <i>ParXtra1</i>
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_NOT_SUPPORTED	The device does not include a modulator

Remarks

Changing the modulation parameters may change the symbol rate!

This is because DTAPI automatically computes the symbol rate from the Transport-Stream rate (as set with **SetTsRateBps**) and the modulation parameters.

DtOutpChannel::SetPower

DTA-102 only. Turn on/off power for a target adapter attached to the DTA-102.

```
DTAPI_RESULT DtOutpChannel::SetPower(
    [in] int    Power           // New power state
);
```

Parameters

Power

New power state according to the table below.

Value	Meaning
DTAPI_POWER_OFF	No power is applied. The 25-pin sub_D connector is compatible with DVB-SPI
DTAPI_POWER_ON	Apply power (+5V) to pin 12 and 25 of the 25-pin sub-D connector

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Power state has been changed successfully
DTAPI_E_INVALID_MODE	The specified power-mode value is invalid
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_NOT_SUPPORTED	The hardware function does not support a power connection for target adapters (DTA-100, DTA-140)

Remarks

After **Attach** and after **Reset**, power is turned off.

DtOutpChannel::SetRfControl

Set up-converter parameters for devices with on-board RF up-converter.

```
DTAPI_RESULT DtOutpChannel::SetRfControl (
    [in] __int64  RfRate      // RF frequency in Hz
);
```

Parameters

RfRate

New carrier frequency for RF up-converter, specified in Hertz. The table below specifies the valid range and the step size with which the RF rate can be specified. *RfRate* is rounded to the nearest RF frequency compatible with the frequency resolution.

Device	Valid Range	Step Size
DTA-107 QPSK	950,000,000 – 2,150,000,000 Hz	200,000 Hz
DTA-110 QAM	400,000,000 – 862,000,000 Hz	250,000 Hz

Result

DTAPI_RESULT	Meaning
DTAPI_OK	The carrier frequency has been changed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_RATE	The specified carrier frequency is incompatible (too low or too high) with the up-converter
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_NOT_SUPPORTED	The device does not have an RF up-converter

Remarks

Changing the RF frequency takes some time to let the PLL settle at the new frequency. The lock status of the PLL may be checked with `GetRfControl`.

DtOutpChannel::SetTsRateBps

Set the channel's *Transport-Stream* rate. Please note that the Transport-Stream rate is always based on 188-byte packets, as defined in the MPEG-2 Systems specification. Therefore, the Transport-Stream rate will be different from the transmit clock for 204-byte transmit modes.

```
DTAPI_RESULT DtOutpChannel::SetTsRateBps (
    [in] int  ClockGenMode,    // Internal or external clock
    [in] int  TsRate           // Transport-Stream rate in bps
);
```

Parameters

ClockGenMode

Clock generator mode.

Value	Meaning
DTAPI_TXCLOCK_INTERNAL	Use internal clock generator
DTAPI_TXCLOCK_EXTERNAL	Use external clock input (DTA-102 only)

TsRate

New Transport-Stream rate specified in bits per second. This parameter is ignored if *ClockGenMode* is **DTAPI_CLOCK_EXTERNAL**.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	The Transport-Stream rate has been changed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_CGMODE	The specified clock-generator mode is invalid or incompatible with the attached hardware function
DTAPI_E_INVALID_RATE	The specified transport-rate is invalid (negative) or incompatible (too high) with the attached hardware function
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

In modes **DTAPI_TXMODE_204** and **DTAPI_TXMODE_ADD16** the transmit-clock rate is set to 204/188 times the specified Transport-Stream clock.

The Transport-Stream rate is usually set in the initialisation phase just after **AttachToSlot**, **AttachToType** or **Reset**. It is recommended to first set the transmit mode with **SetTxMode** before setting the bit rate so that all packets, including the initial ones, are sent in the same format.

SetTsRateBps may also be used while packets are being transmitted. The DTA/U series of devices imposes no constraints on the bit-rate step size or on the number of changes per second. Note however that bit-rate changes may lead to a (temporary) violation of the MPEG-2 Systems requirements on Transport-Streams.

The Transport-Stream rate may be set to zero. This effectively disables packet transmission.

DtOutpChannel::SetTxControl

Set the channel's transmit-control state.

```
DTAPI_RESULT DtOutpChannel::SetTxControl (
    [in] int    TxControl    // Transmit-control state
);
```

Parameters

TxControl

New transmit-control state according to the table below.

Value	Meaning
DTAPI_TXCTRL_IDLE	Execution of data transfers is disabled. No packets from the Transmit FIFO are transmitted. However, the output is stuffed with null packets, except if transmit mode is DTAPI_TXMODE_RAW , in which case no packets are transmitted at all.
DTAPI_TXCTRL_HOLD	Data transfers are enabled. Data can be transferred until the Transmit FIFO is fully loaded. Packet transmission is disabled, in the same way as in state DTAPI_TXCTRL_IDLE .
DTAPI_TXCTRL_SEND	Normal operation. Both data transfers and packet transmission are enabled.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Transmit-control state has been changed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INSUF_LOAD	For DTA-110T only: FIFO load is insufficient to start OFDM modulation
DTAPI_E_INVALID_MODE	The specified transmit-control state is invalid or incompatible with the attached hardware function
DTAPI_E_NO_IPPARS	TS-over-IP channels: IP parameters have not been specified yet
DTAPI_E_NO_IPPARS	For TS-over-IP channels: cannot set transmission state because IP parameters have not been specified yet
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

Setting the transmit-control state can be used for controlled start-up and shutdown of the data-streaming process. In state **DTAPI_TXCTRL_HOLD**, the Transmit FIFO can be loaded with packets (using DMA transfers), while no packets are transmitted yet. Then, if enough *credit* has been built up in

the Transmit FIFO, state **DTAPI_TXCTRL_SEND** is entered. This procedure prevents accidental Transmit-FIFO underflow in the start-up phase.

For TS-over-IP channels, state **DTAPI_TXCTRL_HOLD** or **DTAPI_TXCTRL_SEND** can only be entered if the IP transmission parameters have been specified using **SetIpPars**.

After **AttachToSlot**, **AttachToType** and after **Reset** the transmit-control state is initialised to **DTAPI_TXCTRL_IDLE**.

DtOutpChannel::SetTxMode

Set the channel's transmit mode.

```
DTAPI_RESULT DtOutpChannel::SetTxMode (
    [in] int    TxMode,           // Transmit mode
    [in] int    StuffMode        // Null-packet stuffing on/off
);
```

Parameters

TxMode

New transmit mode according to the table below.

Value	Meaning
DTAPI_TXMODE_188	188-byte mode Packets in the Transmit FIFO are assumed to be 188-bytes long. Packets are transmitted without modification.
DTAPI_TXMODE_192	192-byte mode (DTA-102 only) Packets in the Transmit FIFO are assumed to be 192-bytes long. The SYNC byte of every second packet may be modified (not 0x47). PSYNC is pulsed at the start of every 192-byte packet.
DTAPI_TXMODE_204	204-byte mode Packets in the Transmit FIFO are assumed to be 204-bytes long. Packets are transmitted without modification.
DTAPI_TXMODE_ADD16	Add-16 bytes mode Packets in the Transmit FIFO are assumed to be 188-bytes long. The device adds 16 placeholder bytes (0) to every packet.
DTAPI_TXMODE_RAW	Raw mode No assumptions are made on packet structure. Bytes in the buffer are transmitted unmodified. Null-packet stuffing cannot be applied. This mode is not allowed for TS-over-IP channels.

The following mode is valid for TS-over-IP transmission only.

Value	Meaning
DTAPI_TXMODE_IPRAW	Raw IP mode The Transmit FIFO contains time-stamped IP packets that are transmitted unmodified.

For DVB-ASI output channels *TxMode* can be OR-ed with **DTAPI_TXMODE_BURST**, which indicates that packets should be sent in bursts. If this flag is not specified, packet data is "continuous" (linear over time).

StuffMode

This parameter controls the behaviour of the output when there is no packet data available for transmission from the Transmit FIFO.

If *StuffMode* is '1' (On), the output is stuffed with null packets. The size of inserted null packets is matched to *TxMode*. Packet Stuffing is not supported if *TxMode* is **DTAPI_TXMODE_RAW**, because both packet size and packet boundaries are unknown (**DTAPI_E_MODE** error is returned.)

If *StuffMode* is '0' (Off), null-packet stuffing is not applied. If the Transmit FIFO underflows:

- For DVB-ASI outputs (DTA-100, DTA-140), the output is stuffed with K28.5 characters;
- For DVB-SPI outputs (DTA-102), DVALID is de-asserted.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Transmit mode has been changed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_MODE	The specified transmit mode is invalid or incompatible with the output channel
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function
DTAPI_E_NOT_IDLE	For DTA-110T only: transmit mode can only be changed when transmit-control state is DTAPI_TXCTRL_IDLE

Remarks***Changing the transmit mode may change the transmit-clock rate!***

E.g. if transmit mode is changed from **DTAPI_TXMODE_ADD16** to **DTAPI_TXMODE_188** the transmit-clock is multiplied by 188/204. The **DTAPI** keeps the Transport-Stream rate constant.

The transmit mode is usually set in the initialisation phase just after **AttachToSerial**, **AttachToSlot**, **AttachToType** or **Reset**. It is recommended to set the transmit mode before setting the Transport-Stream rate.

SetTxMode may also be used while packets are being transmitted; no hardware constraints apply. However, an uncontrolled transition effect in the Transport Stream may occur. It is recommended to stop transmission and clear the Transmit FIFO with **ClearFifo** before changing transmit mode.

DtOutpChannel::Write

Write data bytes to the output channel.

```
DTAPI_RESULT DtOutpChannel::Write (
    [in] char*   pBuffer,           // Pointer to data to be written to hardware
    [in] int     NumBytesToWrite // Number of bytes to be written to hardware
);
```

Parameters

pBuffer

Pointer to the buffer containing the data to be written to the output channel. The pointer must be aligned to a 32-bit word boundary.

NumBytesToWrite

Number of bytes to be written to the output channel. The buffer size must be positive and a multiple of four.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Write operation has been completed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_BUF	The buffer is not aligned to a 32-bit word boundary
DTAPI_E_INVALID_SIZE	The specified transfer size is negative or not a multiple of four
DTAPI_E_IDLE	For TS-over-IP channels: cannot write data because transmission-control state is DTAPI_TXCTRL_IDLE
DTAPI_E_NO_TSRATE	For TS-over-IP channels: cannot write data because Transport-Stream rate has not been specified, or is too low
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The data buffer can be any buffer in user space that is aligned to a 4-byte boundary. The device driver locks the physical pages of the buffer in memory. If the pages were swapped to disk, they are read back into memory. Then for PCI Cards a scatter/gather list is created and the DMA transfer is initiated.

Write returns when all data has been transferred to the Transmit FIFO. Note that data is only transferred when the transmit-control state is **DTAPI_TXCTRL_HOLD** or **DTAPI_TXCTRL_SEND** (see [SetTxControl](#)), and if sufficient space is available in the Transmit FIFO.

The thread executing **write** sleeps if one of the following conditions is true:

- Transmit-control state is **DTAPI_TXCTRL_IDLE** (except for TS-over-IP channels; in this case, **write** returns **DTAPI_E_IDLE**).
- Transmit-control state is **DTAPI_TXCTRL_HOLD**, and the number of bytes to be written is greater than the size of the Transmit FIFO.
- Transmit-control state is **DTAPI_TXCTRL_SEND**, and the transmit rate is zero.

The call will be completed if the blocking condition is removed (from another thread).

The old name of this function is `WriteUsingDma`. This latter function name is still defined for backward compatibility.

DtOutpChannel::WriteDirect

This function is for diagnostics purposes only, please use method Write for 'ordinary' writing
Write data bytes to the output channel using "direct" write-cycles to the PCI bus.

```
DTAPI_RESULT DtOutpChannel::WriteDirect (
    [in] char*   pBuffer,           // Pointer to data to be written to hardware
    [in] int     NumBytesToWrite // Number of bytes to be written to hardware
);
```

Parameters

pBuffer

Pointer to the buffer containing the data to be written to the output channel. The pointer must be aligned to a 32-bit word boundary.

NumBytesToWrite

Number of bytes to be written to the output channel. The buffer size must be positive and a multiple of four.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Write operation has been completed successfully
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_INVALID_BUF	The buffer is not aligned to a 32-bit word boundary
DTAPI_E_INVALID_SIZE	The specified transfer size is negative or not a multiple of four
DTAPI_E_FIFO_FULL	The operation timed out because the Transmit FIFO is full
DTAPI_E_NOT_ATTACHED	Channel object is not attached to a hardware function

Remarks

The data buffer can be any buffer in user space that is aligned to a 4-byte boundary.

For USB Devices **WriteDirect** has exactly the same function as **Write**, but for PCI cards **WriteDirect** is considerably slower because the processor writes the data bytes directly to the PCI bus instead of using DMA. **WriteDirect** should only be used for special purposes, e.g. for PCI-card validation.

Unlike **Write**, this function cannot block. If the write operation takes too long, error code **DTAPI_E_FIFO_FULL** is returned.

TsInpChannel

TsInpChannel

Class representing an input channel for receiving an MPEG-2 Transport Stream.

```
class TsInpChannel : public DtInpChannel {  
    :  
    :  
    :  
};
```

Description

Class **TsInpChannel** is a specialisation of the generic **DtInpChannel** class. **TsInpChannel** supports the Transport-Stream functions of **DtInpChannel**. Methods dedicated to SDI will fail.

TsOutpChannel**TsOutpChannel**

Class representing an output channel for transmitting an MPEG-2 Transport Stream.

```
class TsOutpChannel : public DtOutpChannel {  
    :  
    :  
    :  
};
```

Description

Class **TsOutpChannel** is a specialisation of the generic **DtOutpChannel** class. **TsOutpChannel** supports the Transport-Stream functions of **DtOutpChannel**. Methods dedicated to SDI will fail.

DtLoop**DtLoop**

DtLoop class represents an object that provides functionality to loop MPEG-2 transport-streams from a DtInpChannel to a DtOutpChannel object.

```
class DtLoop;
```

DtLoop::AttachToInput

Attach a `DtInpChannel` object to the loop object.

```
DTAPI_RESULT DtLoop::AttachToInput (
    [in] DtInpChannel*  pDtInp          // Input channel to attach
);
```

Parameters

pDtInp

Pointer to the `DtInpChannel` object to be used as input by the `DtLoop` object. The `DtInpChannel` object, passed here, must stay in scope until the object is detached from the `DtLoop` object.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Loop object has been attached successfully to input channel
DTAPI_E_ATTACHED	Loop object is already attached to a other input channel
DTAPI_E_INVALID_ARG	Invalid <i>pDtInp</i> pointer (i.e. NULL-pointer)
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	The <i>pDtInp</i> object has not been attached to any hardware input function

Remarks

After attaching a `DtInpChannel` object to a `DtLoop` object the `DtInpChannel` object may still be used to apply certain settings like setting the IP-over-TS parameters (`DtInpChannel::SetTsIpPars`) or getting the Ts-Rate (`DtInpChannel::GetTsRateBps`). However note that certain methods on the `DtInpChannel` object will have limited functionality or may not be called while the object is attached to a `DtLoop` object. For example the `DtInpChannel::Read` method may not be called while attached to a `DtLoop` object.

DtLoop::AttachToOutput

Attach a `DtOutputChannel` object to the loop object.

```
DTAPI_RESULT DtLoop::AttachToOutput (
    [in] DtOutputChannel* pDtOutp      // Output channel to attach
);
```

Parameters

pDtOutp

Pointer to the `DtOutputChannel` object to be used as input by the `DtLoop` object. The `DtOutputChannel` object, passed here, should stay in scope until the object is detached from the `DtLoop` object.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Loop object has been attached successfully to input channel
DTAPI_E_ATTACHED	Loop object is already attached to a other input channel
DTAPI_E_INVALID_ARG	Invalid <i>pDtOutp</i> pointer (i.e. NULL-pointer)
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver
DTAPI_E_NOT_ATTACHED	The <i>pDtOutp</i> object has not been attached to any hardware input function

Remarks

After attaching a `DtOutputChannel` object to a `DtLoop` object the `DtOutputChannel` object may still be used to apply certain settings like setting the IP-over-TS parameters (`DtOutputChannel::SetTsIpPars`). However note that certain methods on the `DtOutputChannel` object will have limited functionality or may not be called while the object is attached to a `DtLoop` object. For example the `DtOutputChannel::Write` method may not be called while attached to a `DtLoop` object.

DtLoop::Detach

Detaches the loop object from the associated `DtInpChannel` and `DtOutpChannel` objects.

```
DTAPI_RESULT DtLoop::Detach();
```

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Loop object has been detached successfully
DTAPI_E_NOT_ATTACHED	Loop object was not attached to any input or output
DTAPI_E_NOT_IDLE	Cannot detach while the loop object is started

Remarks

DtLoop::DetachFromInput

Detaches the loop object from the associated `DtInpChannel` object

```
DTAPI_RESULT DtLoop::DetachFromInput ();
```

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Loop object has been detached successfully
DTAPI_E_NOT_ATTACHED	Loop object was not attached to any input
DTAPI_E_NOT_IDLE	Cannot detach while the loop object is started

Remarks

DtLoop::DetachFromOutput

Detaches the loop object from the associated `DtOutputChannel` object

```
DTAPI_RESULT DtLoop::DetachFromOutput ();
```

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Loop object has been detached successfully
DTAPI_E_NOT_ATTACHED	Loop object was not attached to any output
DTAPI_E_NOT_IDLE	Cannot detach while the loop object is started

Remarks

DtLoop::IsStarted

Returns true if the looping from input-to-output has started.

```
bool DtLoop::IsStarted ();
```

Result

Result	Meaning
true	Looping has started
false	Looping has not started

Remarks

DtLoop::SetStuffingMode

Set NULL-packet stuffing mode parameters.

```
DTAPI_RESULT DtLoop::SetStuffingMode (
    [in] int    Mode           // Stuffing mode
    [in] int    TsRate        // Output rate
);
```

Parameters

Mode

New stuffing mode according to the table below.

Value	Meaning
DTAPI_STUFFMODE_OFF	Stuffing mode should be disabled
DTAPI_STUFFMODE_ON	Stuffing mode should be enabled

TsRate

Specifies the desired output Ts-Rate. The **DtLoop** object will automatically add/remove NULL-packets to/from the received transport-stream to reach the rate specified here.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Remux mode has been changed successfully
DTAPI_E_INVALID_MODE	The specified stuffing mode is invalid
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver

DtLoop::Start

Start/Stop looping from input-to-output.

```
DTAPI_RESULT DtLoop::Start (
    [in] bool    Start=true        // Start looping
);
```

Parameters

Start

Set to true too start looping. Set to false stop looping.

Result

DTAPI_RESULT	Meaning
DTAPI_OK	Looping has started successfully
DTAPI_E_NOT_ATTACHED	Loop-controller object has not been attached to an input and/or output
DTAPI_E_DEV_DRIVER	Unclassified failure in device driver

Remarks

Before calling this method the loop-controller object must have been attached to an input and output through successful calls to **DtLoop::AttachToInput** and **DtLoop::AttachToOutput**

4. Ports and Hardware Functions per Device

Table 6. PCI Cards – Ports and Hardware Functions

Device	Port	Index	ChanType	StreamType	Capabilities	Remark
DTA-100	1	0	OUTPUT	ASI_SDI	CAP_ASI	Doubly buffered
DTA-102	1	0	OUTPUT	TS_SPI		
DTA-107	1	0	OUTPUT	TS_MOD	CAP_QPSK CAP_LBAND	Doubly buffered
DTA-110	1	0	OUTPUT	TS_MOD	CAP_QAM CAP_UHF	Doubly buffered
DTA-110T	1	0	OUTPUT	TS_MOD	CAP_OFDM CAP_QAM CAP_UHF	Doubly buffered
DTA-120	1	0	INPUT	ASI_SDI	CAP_ASI	
DTA-122	1	0	INPUT	TS_SPI		
DTA-124	1..4	0..3	INPUT	ASI_SDI	CAP_ASI CAP_SDI	
DTA-140	1	0	INPUT	ASI_SDI	CAP_ASI	
	2	0	OUTPUT	ASI_SDI	CAP_ASI	Doubly buffered
DTA-145	1	0	INPUT	ASI_SDI	CAP_ASI CAP_BIDIR CAP_SDI	Programmable input/output
		0	OUTPUT			
	2	0/1	OUTPUT	ASI_SDI	CAP_ASI CAP_SDI	
DTA-160	1..3	0..2	INPUT	ASI_SDI	CAP_ASI CAP_BIDIR CAP_SDI	Programmable input/output
		0..2	OUTPUT			
	4	0	INPUT OUTPUT	TS_OVER_IP		Up to 20 parallel streams (10 input + 10 output)